





UNIVERSITÉ DE FIANARANTSOA

ÉCOLE DE MANAGEMENT ET D'INNOVATION TECHNOLOGIQUE (EMIT)

Mention : INFORMATIQUE

Parcours : Développement d'Application Intranet – Internet (D.A.2.I)

MÉMOIRE DE FIN DE CYCLE

Pour l'obtention du Diplôme de Licence en Informatique

CONCEPTION ET REALISATION D'UNE PLATEFORME DE SUIVI DES PROJETS ET PARTENARIATS DE L'UNIVERSITE DE FIANARANTSOA

(Volet frontend : interface utilisateur, ergonomie et intégration de l'API)

Présenté par : RAOTOMIARINA Iriela

Encadreur : Dr RAKOTOARISON Tsiorinantenaina

Evaluateur : Mr MAMINIAINA Indrafo Henri

Année universitaire : 2025–2026







UNIVERSITÉ DE FIANARANTSOA

ÉCOLE DE MANAGEMENT ET D'INNOVATION TECHNOLOGIQUE (EMIT)

Mention : INFORMATIQUE

Parcours : Développement d'Application Intranet – Internet (D.A.2.I)

MÉMOIRE DE FIN DE CYCLE

Pour l'obtention du Diplôme de Licence en Informatique

CONCEPTION ET REALISATION D'UNE PLATEFORME DE SUIVI DES PROJETS ET PARTENARIATS DE L'UNIVERSITE DE FIANARANTSOA

(Volet frontend : interface utilisateur, ergonomie et intégration de l'API)

Présenté par : RAOTOMIARINA Iriela

Encadreur : Dr RAKOTOARISON Tsiorinantenaina

Evaluateur : Mr MAMINIAINA Indrafo Henri

Année universitaire : 2025–2026

CURRICULUM VITAE

RAOTOMIARINA Iriela

DEVELOPPEUR WEB ET MOBILE

Contact

iraotomiarina@gmall.com
+261 38 35 852 04
Lot IVE 85 Imerintsiatosika Antananarivo
Iriela Matthieu RAOTOMIARINA

Étudiant en troisième année de Licence en Développement d'Applications Intranet et Internet, avec de solides bases en développement front-end, back-end et gestion de bases de données. Curieux, autonome et rigoureux.

Formation

BACC Série D 2023 :

- **Lycée Fo Masin'I Jesoa Imerintsiatosika**

Première année de licence 2023 - 2024 :

- **EMIT FIANARANTSOA**

Deuxième année de licence 2024 – 2025 :

- **EMIT FIANARANTSOA**

Troisième année de licence 2025 – 2026 :

- **EMIT FIANARANTSOA**

Langues

- Malagasy : Langue maternelle
- Français : Bien
- Anglais : Bien

Environnement et outil

- VS Code
- Android Studio
- IntelliJ IDE
- Git / GitHub
- Docker
- Postman
- Cisco Packet Tracer
- GNS 3
- Virtual Box

Qualités

- Organisé
- Dynamique
- Polyvalent
- Ponctuel
- À l'écoute
- Réactive
- Esprit d'équipe

Expériences professionnelles

STAGE DE DEUX MOIS | Juin 2025 – Aout 2025
Préfecture de Fianarantsoa

- Virtualisation d'infrastructure réseau avec Virtual Box
- Gestion et configuration des environnements virtuels.

PROJET ACADEMIQUE | 2023 – 2026
EMIT FIANARANTSOA

Troisième année de licence :

- GitHelper : Application mobile d'assistance Git avec visualisation de branche et historique
- Plateforme e-learning
- Inscription au cours d'auto-école
- Analyse de ventes : de la collecte des données à la prédiction (Data science)
- Gestion des Salles et emplois du temps
- Plateforme collaborative de partage de compétences

Deuxième année de licence :

- Location de voiture
- Vente de téléphone portable
- Gestion de location (Application lourde)

Première année de licence :

- Gestion de messe
- Location de voiture
- Vente de téléphone

Langage, Framework et Base de données

- Dart / Flutter
- Java / SpringBoot
- C # / .NET
- Python / Django
- JavaScript / Node.js
- HTML /CSS / Bootstrap / Tailwind
- PHP / Laravel / Codeigniter
- Firebase
- Postgres SQL
- MySQL/ Xampp / Wamp

REMERCIEMENTS

La réalisation de ce rapport de stage a nécessité l'accompagnement, la disponibilité et le soutie n de plusieurs personnes. Leurs conseils, leurs encouragements et leurs différentes contributio ns ont constitué une aide précieuse tout au long de mon stage et de la rédaction du présent rap port. Je tiens donc à exprimer mes sincères remerciements à toutes les personnes qui ont contr ibué, de près ou de loin, au bon déroulement de mon stage ainsi qu'à l'aboutissement de ce tra vail.

J'adresse particulièrement mes remerciements :

- à Monsieur le Président de l'Université de Fianarantsoa, le Professeur HAJALALAINA Aim é Richard, pour son engagement en faveur de la formation universitaire ;

- à Madame la Docteure FANJANIAINA Marie Lucia, Directrice de l'Équipe Projets et Parten ariats de la Présidence de l'Université de Fianarantsoa, ainsi qu'à l'ensemble des membres de s on équipe, pour la confiance qu'ils m'ont accordée en me permettant de concevoir une solution adaptée à leurs besoins, pour leur disponibilité et pour leur collaboration tout au long de ce pr ojet ;

- au Professeur RAKOTONIRAINY Hasina Lalaina, Directeur de l'École de Management et d 'Innovation Technologique (EMIT), pour m'avoir accueilli au sein de cet établissement et m'a voir permis de poursuivre mes études ;

- au Docteur TAREHY Brice Evrard, Responsable de la Mention Développement d'Applicatio ns Intranet et Internet (DAII), pour son accompagnement tout au long de mon parcours univer sitaire ;

- au Docteur HERY ANTENAINA Paul Franck, Directeur Gérant de ITDC MADA, pour m'a voir accueilli au sein de son entreprise et m'avoir donné l'opportunité d'effectuer mon stage da ns un environnement professionnel ;

- au Docteur RAKOTOARISON Tsiorinantenaina, mon encadreur professionnel, pour sa disp onibilité, ses conseils, son accompagnement et le partage de ses connaissances tout au long de mon stage ;

- à l'ensemble de l'équipe informatique de ITDC MADA, ainsi qu'à mes collègues, pour leur a ccueil, leur collaboration, leur esprit d'équipe et leur soutien durant cette période de stage.

RÉSUMÉ

Ce mémoire présente la conception et la réalisation d'une plateforme web de suivi des projets et des partenariats de l'Université de Fianarantsoa, développée en binôme au cours d'un stage de trois mois. La plateforme vise à centraliser le suivi précédemment réalisé sous Excel et comprend un espace interne organisé autour de trois rôles (Présidence, Établissement et Décideur), ainsi qu'un espace public. Elle repose sur une architecture en trois niveaux : un frontend React et TypeScript, une API REST NestJS et une base de données PostgreSQL. Le développement a suivi une démarche Agile, et UML a servi à la modélisation. Le mémoire porte principalement sur la contribution frontend, notamment la gestion des routes par rôle, des sessions, des formulaires validés, des tableaux de bord et des tests automatisés.

Mots-clés : plateforme web, suivi de projets, partenariats, conventions, React, TypeScript, NestJS, UML, Agile.

LISTE DES FIGURES

Figure 1.1 : Organigramme de l'EMIT 10
Figure 2.1 : Méthode de travail d'ITDC Mada 13
Figure 2.2 : Organigramme d'ITDC Mada 14
Figure 3.1 : Planification du projet (diagramme de Gantt) 19
Figure 4.1 : Diagramme de flux de l'organisation actuelle 21
Figure 4.2 : Place de l'approche Agile dans le déroulement du projet 24
Figure 5.1 : Diagramme de cas d'utilisation général 31
Figure 5.2 : Diagramme de séquence système du cas « Créer une convention » 33
Figure 5.3 : Modèle de domaine (vue d'ensemble) 35
Figure 6.1 : Architecture logique de la plateforme 36
Figure 6.2 : Diagramme de séquence de l'authentification 39
Figure 6.3 : Diagramme de séquence de la création d'une convention 41
Figure 6.4 : Diagramme d'activités du circuit de traitement d'une convention 42
Figure 6.5 : Diagramme d'états d'une publication 43
Figure 6.6 : Diagramme d'états d'une convention 44
Figure 6.7 : Diagramme d'états d'un projet 44
Figure 6.8 : Diagramme de classes du domaine « Projets » 45
Figure 6.9 : Diagramme de classes du domaine « Partenaires, partenariats et cotutelles » 46
Figure 6.10 : Diagramme de classes du domaine « Conventions, documents et publications » 46
Figure 6.11 : Diagramme de paquetage du frontend 47
Figure 6.12 : Diagramme de déploiement logiciel (Docker Compose) 48
Figure 8.1 : Page d'accueil de l'espace public 60
Figure 8.2 : Conteneurs Docker de l'environnement de développement 61
Figure 8.3 : Page de connexion 62
Figure 8.4 : Tableau de bord de la Présidence 62
Figure 8.5 : Liste des conventions et formulaire de création 63
Figure 8.6 : Tableau de bord d'un établissement 63
Figure 8.7 : Assistant de création d'un projet 64
Figure 8.8 : Tableau de bord du Décideur 64
Figure 8.9 : Synthèse annuelle par établissement 65
Figure 8.10 : Tiroir de notifications 66

LISTE DES TABLEAUX

Tableau 1.1 : Mentions et parcours de l'EMIT 4
Tableau 3.1 : Acteurs et droits d'accès 16
Tableau 3.2 : Besoins fonctionnels principaux 17
Tableau 3.3 : Répartition des responsabilités au sein du binôme 18
Tableau 4.1 : Limites du suivi sous Excel au regard des exigences 21
Tableau 4.2 : Comparaison des solutions envisagées 22
Tableau 4.3 : Diagrammes UML retenus pour le projet 25
Tableau 4.4 : Comparaison des options pour le frontend 26
Tableau 4.5 : Outils et bibliothèques du frontend 26
Tableau 4.6 : Outils du backend 27
Tableau 5.1 : Classes du modèle par domaine 29
Tableau 5.2 : Énumérations du modèle 29
Tableau 5.3 : Besoins fonctionnels par acteur 30
Tableau 5.4 : Cas d'utilisation « Créer une convention » 32
Tableau 5.5 : Cas d'utilisation « S'authentifier » 32
Tableau 5.6 : Relations principales du modèle 34
Tableau 6.1 : Correspondance entre exigences et choix d'architecture 37
Tableau 8.1 : Conteneurs de l'environnement de développement 61
Tableau 9.1 : Confrontation des objectifs et des résultats 67
Tableau 9.2 : Tests automatisés présents dans le dépôt 68

LISTE DES ABRÉVIATIONS

Cette liste regroupe les abréviations utilisées dans le mémoire, classées par ordre alphabétique.

- **API** — Application Programming Interface (interface de programmation d'applications)
- **ARIA** — Accessible Rich Internet Applications (attributs d'accessibilité)
- **CORS** — Cross-Origin Resource Sharing (partage de ressources entre origines)
- **CSS** — Cascading Style Sheets (feuilles de style en cascade)
- **DA2I** — Développement d'Application Internet – Intranet
- **DOM** — Document Object Model (modèle objet de document)
- **DTO** — Data Transfer Object (objet de transfert de données)
- **E2E** — End-to-End (de bout en bout)
- **EMIT** — École de Management et d'Innovation Technologique
- **FK** — Foreign Key (clé étrangère)
- **HTML** — HyperText Markup Language
- **HTTP** — HyperText Transfer Protocol
- **JSON** — JavaScript Object Notation
- **JSX** — JavaScript XML
- **JWT** — JSON Web Token (jeton web JSON)
- **KPI** — Key Performance Indicator (indicateur clé de performance)
- **OMG** — Object Management Group
- **ORM** — Object-Relational Mapping (mappage objet-relationnel)
- **PDF** — Portable Document Format
- **PME** — Petites et moyennes entreprises
- **QCD** — Qualité, coûts, délais
- **REST** — Representational State Transfer
- **RG** — Règle de gestion
- **SGBD** — Système de gestion de base de données
- **SIG** — Système d'information géographique
- **SMTP** — Simple Mail Transfer Protocol
- **SPA** — Single Page Application (application monopage)
- **SQL** — Structured Query Language (langage de requête structuré)

- **TDR** — Termes de référence
- **TS** — TypeScript
- **TSX** — TypeScript XML
- **UF** — Université de Fianarantsoa
- **UML** — Unified Modeling Language (langage de modélisation unifié)
- **URL** — Uniform Resource Locator
- **UUID** — Universally Unique Identifier (identifiant unique universel)
- **XLSX** — Format de classeur Excel (Office Open XML)

GLOSSAIRE

Les termes ci-dessous sont définis tels qu'ils sont employés dans le présent mémoire.

- **Activité** — Tâche d'un projet, avec un intitulé, un état (terminée ou non) et éventuellement une échéance. L'avancement du projet doit être calculé à partir des activités terminées (exigence non implémentée dans cette version).
- **Archivage** — Conservation d'un enregistrement ou d'une ancienne version sans suppression définitive. L'enregistrement reste consultable.
- **Authentification** — Vérification de l'identité d'un utilisateur (identifiant, mot de passe, jetons JWT) avant l'accès à l'espace interne.
- **Backend** — Partie serveur de l'application : base de données, API, logique métier et sécurité. Dans ce projet, elle est développée avec NestJS.
- **Bailleur** — Organisme qui finance un projet.
- **Bénéficiaire** — Personne ou groupe qui profite des résultats d'un projet.
- **Brouillon** — Statut d'un enregistrement en cours de saisie, ou, pour une publication, non encore rendue publique.
- **Chronogramme** — Enregistrement, pour une activité, de l'année, du trimestre, du mois et du nombre de jours consacrés. Il est indépendant du calcul de l'avancement.
- **Composant** — Brique d'interface réutilisable (bouton, tableau, formulaire) dans React.
- **Convention** — Accord formalisé entre l'Université et un partenaire, avec ses dates et son statut de signature. Elle peut être liée à un partenariat, à un projet, à une cotutelle, à plusieurs de ces entités, ou à aucune.
- **Cotutelle** — Collaboration académique autour d'une thèse, associée à un doctorant et à une université partenaire.
- **Établissement** — Structure de l'Université qui porte des projets et des partenariats. Chaque établissement dispose d'un compte unique.
- **Frontend** — Partie visible de l'application, avec laquelle l'utilisateur interagit dans le navigateur. Dans ce projet, elle est développée avec React et Vite.
- **Hook** — Fonction React qui gère l'état ou les effets d'un composant fonctionnel (par exemple le hook d'authentification, qui fournit l'utilisateur connecté).
- **Migration** — Script versionné qui fait évoluer le schéma de la base de données (ajout d'une colonne, par exemple).

- **Notification** — Message interne signalant un événement (ajout, échéance, expiration de convention, retard), avec un état lu ou non lu.
- **Partenaire** — Organisme extérieur (entreprise, institution, association, université) avec lequel l'Université établit une collaboration.
- **Partenariat** — Relation de collaboration entre un ou plusieurs établissements et un ou plusieurs partenaires, pouvant regrouper plusieurs projets.
- **Projet** — Action portée par un établissement, avec son statut, ses dates, son budget, ses activités et ses échéances.
- **Publication** — Contenu destiné à l'espace public, distinct du projet, comprenant un titre, une description publique et des photos. Elle n'est visible des visiteurs qu'au statut « Publiée ».
- **Rôle** — Niveau d'accès d'un compte. La plateforme en distingue trois : Présidence, Établissement et Décideur.
- **Synthèse annuelle** — Vue consolidée de l'activité de coopération, calculée automatiquement par établissement et complétée par des éléments saisis manuellement.
- **Wizard** — Formulaire découpé en étapes successives, avec indicateur de progression et sauvegarde du brouillon.

SOMMAIRE

INTRODUCTION GÉNÉRALE 1
PARTIE 1 : PRÉSENTATION GÉNÉRALE 2
Chapitre 1 : Ecole de Management et d'Innovation Technologique 3
1.1. Présentation générale 3
1.2. Historique 3
1.3. Missions et formations 4
1.4. Organisation 9
Chapitre 2 : Présentation ITDC Mada 11
2.1. Historique 11
2.2. Services 11
2.3. Formations 12
2.4. Mission 12
2.5. Clients de l'ITDC MADA 12
2.6. Méthode de travail 13
2.7. Organigramme d'ITDC MADA 14
Chapitre 3 : Présentation du projet 15
3.1. Formulation du projet 15
3.2. Justification de l'importance du sujet 15
3.3. Objectifs et besoins utilisateurs 16
3.4. Moyens nécessaires à la réalisation du projet 18
3.5. Chronogramme de réalisation du projet 19
3.6. Résultat attendu 19
PARTIE 2 : ANALYSE ET CONCEPTION 20
Chapitre 4 : Analyse préalable 21
4.1. Analyse de l'existant 21
4.2. Choix de la solution 22

4.3. Choix de la méthode de conception et de gestion de projet 23
4.4. Choix de l'outil de conception 25
4.5. Choix du langage de programmation 25
4.6. Choix du framework 26
4.7. Choix de l'environnement de développement 26
4.8. Choix du système de gestion de base de données 28
Chapitre 5 : Conception du projet 29
5.1. Présentation de la méthode de conception 29
5.2. Dictionnaire des données 29
5.3. Règles de gestion 30
5.4. Diagramme de cas d'utilisation 30
5.5. Description textuelle des cas d'utilisation 32
5.6. Diagramme de séquence système 32
5.7. Modèle de domaine 34
Chapitre 6 : Conception détaillée 36
6.1. Architecture de l'application 36
6.2. Diagrammes de séquence de conception 38
6.3. Diagramme de classes 45
6.4. Diagramme de paquetage 47
6.5. Diagramme de déploiement 47
PARTIE 3 : MISE EN ŒUVRE ET RÉSULTATS 49
Chapitre 7 : Mise en œuvre 50
7.1. Architecture du système 50
7.2. Configuration des outils de conception 51
7.3. Génération de la base de données 51
7.4. Configuration de l'environnement de développement 51
7.5. Extraits de code des fonctionnalités principales 51

7.6. Difficultés rencontrées 55
Chapitre 8 : Présentation de l'application 57
8.1. Espace public 57
8.2. Environnement matériel et déploiement logiciel 60
8.3. Authentification 61
8.4. Espace Présidence 62
8.5. Espace Établissement 63
8.6. Espace Décideur 64
8.7. Synthèse annuelle, notifications et exports 65
8.8. Responsive design, thème et accessibilité 66
Chapitre 9 : Évaluation et suggestions 67
9.1. Évaluation de l'application 67
9.2. Contributions académiques et professionnelles 69
9.3. Limitations de l'étude et pistes de recherche futures 69
CONCLUSION GÉNÉRALE 70
Bilan 70
Perspectives 70
BIBLIOGRAPHIE 71
WEBOGRAPHIE 72
ANNEXES 73
Annexe A : Synthèse des règles de gestion 73
Annexe B : Extraits de code complémentaires 74
Annexe C : Résultats d'exécution des tests 74

INTRODUCTION GÉNÉRALE

La transformation numérique des établissements d'enseignement supérieur ne se limite plus à l'informatisation de la scolarité. Elle touche aussi le pilotage de l'activité institutionnelle : suivi des projets, gestion des conventions et des partenariats, valorisation des actions menées auprès du public. La qualité de l'information disponible conditionne la capacité d'une université à rendre compte de son activité, à nouer des coopérations durables et à orienter ses décisions. À l'Université de Fianarantsoa, ce suivi était réalisé sous Excel, ce qui limitait la centralisation, la traçabilité et la séparation des données entre établissements.

Le problème se pose ainsi : les établissements doivent enregistrer leurs propres partenaires, partenariats et projets, tandis que l'équipe Projet et Partenariat de la Présidence doit disposer d'une vision globale, conserver les documents, archiver les conventions et produire une synthèse annuelle. Les progiciels de gestion de projet ou de conventions répondent rarement, de façon complète, à cette organisation en établissements. La question centrale est donc : comment concevoir et réaliser une plateforme web qui permette de suivre projets et partenariats de manière centralisée, sécurisée et adaptée à chaque catégorie d'utilisateurs ?

Le travail s'appuie sur l'hypothèse suivante : une application web à trois espaces (Présidence, établissements, Décideur), avec séparation stricte des données par établissement, répond mieux aux besoins qu'un tableur. Le travail a été réalisé en stage, pendant trois mois, en binôme : le frontend a été confié à l'auteur, le backend au binôme. Il porte sur la conception et la réalisation de la partie frontend, en intégrant les éléments de l'API nécessaires à sa compréhension.

Le mémoire est organisé en trois parties. La première présente l'École, l'entreprise d'accueil et le projet. La deuxième traite l'analyse préalable et la conception, avec le langage UML. La troisième présente la mise en œuvre, l'application réalisée et son évaluation, avant la conclusion générale, la bibliographie, la webographie et les annexes.

PARTIE 1 : PRÉSENTATION GÉNÉRALE

Chapitre 1 : Ecole de Management et d'Innovation Technologique

1.1. Présentation générale

L'Ecole de Management et d'Innovation Technologique (EMIT) est une grande école publique au sein de l'Université de Fianarantsoa, qui a basculé totalement vers le système L.M.D.

L'EMIT nous a reconnu comme étudiants.

1.2. Historique

L'EMIT est une école universitaire publique pluridisciplinaire, rattachée à l'Université de Fianarantsoa. La grande maturité au niveau de l'enseignement et la compétence des étudiants sortants de l'établissement ont permis aux dirigeants sous l'approbation du Ministère de mettre en place la conversion du Centre en Ecole au sein de l'Université de Fianarantsoa par le Décret N°2016-1394 du 15 Novembre 2016. L'EMIT prépare d'une part le diplôme de Master en deux mentions selon trois parcours et d'autre part le diplôme de Licence en trois mentions selon cinq parcours.

Auparavant, elle était connue sous le nom du Centre Universitaire de Formation

Professionnalisant (CUFP), créée par le Décret N°2005-205 du 26 Avril 2005 et dispensait le diplôme de Licence professionnelle en Administration ainsi qu'en Informatique. Mais avant cela, elle était connue également sous le nom du Centre de Formation Continue (CFC), créé par l'Arrêté Rectoral N°99-23/UF/R du 10 Mars 1999 qui formait de diplôme de Technicien Supérieur.

L'EMIT a été sélectionnée « Meilleur Etablissement » pendant le Salon de la Recherche organisé par l'Organisation Internationale du Travail les 5 et 6 Juillet 2017. Depuis l'année universitaire 2013-2014, l'école est basculée totalement vers le système Licence, Master et Doctorat (LMD). Toutes les offres de formation dispensée à l'EMIT sont habilitées par le Ministère de l'Enseignement Supérieur et de la Recherche Scientifique. L'école propose neuf parcours répartis en trois mentions, Présentes ci-dessous :

Tableau 1.1 : Mentions et parcours de l'EMIT

<table>
<tr><td>Cycle</td><td>Mention Management</td><td>Mention Informatique</td><td>Mention Relations Publiques et Multimédia</td></tr>
<tr><td>Licence</td><td>Administration Economique et</td><td>Développement d’Application Internet/Intranet</td><td>Communication Multimédia</td></tr>
<tr><td rowspan="2"></td><td rowspan="2">Sociale</td><td colspan="2"></td></tr>
<tr><td>Conception, Intégration et Gestion des Systèmes d’Information</td><td>Relations Publiques et Communication Organisationnelle</td></tr>
<tr><td rowspan="4">Master</td><td>Management Décisionnel (MD)</td><td>Ingénierie Géo spatiale et Technologies de l’Information (IGTI)</td><td>Communication Multimédia (CM)</td></tr>
<tr><td rowspan="3">Management d’Entreprises et Développement des Affaires (MEDA)</td><td>Modélisation et Ingénierie Informatique (M2I)</td><td>Relations Publiques et Communication Organisationnelle (RPCO)</td></tr>
<tr><td>Système d’Information, Géomatique et Décision (SIGD)</td><td rowspan="2">Communication Numérique et Management de Projet(CNMP)</td></tr>
<tr><td>Sciences des Données et Intelligence Artificielle (SDIA)</td></tr>
</table>

Source : Ecole de Management et d’Innovation Technologique

1.3. Missions et formations

1.3.1. Missions

L’école a pour mission, d’abord de dispenser des formations initiales et continues en informatique, en administration et en relations publiques et multimédia. Ensuite, elle offre des services connexes à l’informatique. Puis, elle forme des techniciens opérationnels immédiatement au sein des entreprises. Enfin, elle assure le perfectionnement professionnel des étudiants, des demandeurs d’emploi, des employés et des cadres d’entreprises.

1.3.2. Formations existantes

L'école présente actuellement deux cycles : Licence et Master. Chaque cycle possède plusieurs parcours assurés par un chef de mention.

- **1.3.2.1. Cycle Licence**

L'école possède trois (03) mentions pour cinq (05) parcours en cycle Licence. Les étudiants toutes mentions confondues doivent en faire, un stage de réalisation en entreprise avec un rapport de stage soutenu pour les étudiants en deuxième année de Licence (L2) et un stage de fin d'études suivi de la soutenance d'un mémoire pour les étudiants en troisième année de Licence (L3). La formation dure trois années universitaire et à la fin de la formation, les étudiants obtiennent des diplômes de Licence.

- **Mention « Management », Parcours « Administration Economique et Sociale (AES) »**

La condition d'accès en première année de Licence se fait par voie de concours pour les titulaires d'un Baccalauréat général de toutes séries (A, C, D, S, L, OSE) ou d'un

Baccalauréat technique G1 et G2. A l'issue de la formation, les étudiants ont les compétences de :

- Assister le Directeur Général, le Directeur des Ressources Humaines et le Directeur
- Administratif et Financier ;
- Gérer les Ressources Humaines ;
- Gérer une entreprise ou un projet.

- **Mention « Informatique », Parcours « Développement d'Application Internet Intranet (DA2I) » et Parcours « Conception, Intégration et Gestion des Systèmes Information (CIGSI) »**

La condition d'accès en première année de Licence se fait par voie de concours pour les élèves titulaires du diplôme de Baccalauréat Général Scientifique (Série C, série D et série S), Baccalauréat Technique Professionnelle et Baccalauréat Technique Technologique (Filière Industrielle, Maintenance Automobile, Ouvrage Bois, Ouvrage Métallique, Génie Civile). A l'issue de la formation, les étudiants sont compétents en :

- Administration des bases de données ;
- Administration des réseaux et systèmes informatiques ;
- Développement d'application client/serveur

Mention « Relations Publiques et Multimédia », Parcours « Communication Multimédia (CM) » et « Relations Publiques et Communication Organisationnelle (RPCO) »

La condition d'accès en première année de Licence se fait par voie de concours pour les titulaires d'un Baccalauréat général de toutes séries (A, C, D, S, L, OSE) ou d'un Baccalauréat technique G1 et G2. A l'issue de la formation, les étudiants ont les compétences de :

- Rédiger un article dans un journal ;
- Occuper un poste d'un technicien de presse ; - Travailler dans la revue de presse.

- **1.3.2.2. Cycle Master**

L'Ecole de Management et d'Innovation Technologique (EMIT) possède trois (03) mentions pour neuf (09) parcours en Master Professionnel ainsi qu'en Master Recherche. La durée de la formation est de quatre semestres c'est-à-dire deux années universitaires.

- **Mention Management, Parcours Management Décisionnel (MD)**

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Administration Economique et Sociale, en Gestion ou en Economie.

Le master en management décisionnel propose une formation approfondie et pluridisciplinaire, façonnant les étudiants dans les sphères de l'administration privée et publique. En mettant l'accent sur la stratégie décisionnelle, sur l'analyse de l'environnement d'intervention, la gestion des risques, les diplômés sont armés pour aborder des défis complexes. L'expertise des étudiants est fortement sollicitée dans divers secteurs, allant de la gestion d'entreprise à l'administration publique, où ils jouent un rôle crucial dans la prise de décision. En combinant une compréhension approfondie des disciplines et des compétences

aiguisées en matière de décision, ce master offre une contribution significative à la réussite des organisations dans un environnement dynamique.

- **Management d'Entreprises et Développement des Affaires (MEDA)**

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Administration Economique et Sociale, en Gestion ou en Economie.

Le master en management d'entreprises et développement des affaires prépare les étudiants à exceller dans la création et la constitution d'entreprises. Les diplômés acquièrent des compétences clés pour élaborer des plans d'affaires solides, identifier les opportunités de marché et gérer efficacement les ressources. Les étudiants sont bien positionnés pour fonder leur propre entreprise en tant qu'entrepreneurs prospères ou occuper des rôles clés dans des start-ups. Les débouchés incluent la création d'entreprises innovantes, la gestion de projets entrepreneuriaux et la consultation stratégique pour les nouvelles initiatives commerciales. Ce master offre ainsi une voie dynamique vers l'entreprenariat et le succès dans le domaine des affaires.

- **Mention « Informatique », Parcours « Système d'Information, Géomatique et Décision (SIGD) » et Parcours « Modélisation et Ingénierie Informatique (M2I) »**

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Informatique et en Mathématique et Informatique pour les Sciences Sociales (MISS). Le Recrutement en S9 se fait par validation des crédits acquis.

Le parcours « Système d'Information, Géomatique et Décision » a pour objectif de donner un panorama des recherches actuelles et émergeantes en terme de système d'aide à la décision. En effet, les systèmes informatiques et la géomatique sont en plein essor, par les grilles de calcul et les multiples appareils mobiles intégrant des systèmes informatiques de plus en plus performants et complexes. Ces systèmes informatiques intégrant un parallélisme massif ou /et une mobilité des composants représentent un défi pour le génie logiciel qui doit fournir de nouvelles méthodes et des outils de production de logiciel pour la description de l'architecture

de ces systèmes complexes et pour leur validation et/ou certification. De plus, l'effervescence des techniques en géomatique qui fournissent des données spatiales et temporelles dans différents domaines représente des moyens efficaces pour prendre les bonnes décisions.

- **Mention « Informatique », Parcours « Ingénierie Géo spatiale et Technologies de l'Information (IGTI) » et Parcours « Sciences des Données et Intelligence Artificielle (SDIA) »**

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Informatique et en Mathématique et Informatique pour les Sciences Sociales (MISS). Le Recrutement en S9 se fait par validation des crédits acquis.

Ce parcours vise à former des experts en informatique de haut niveau, spécialisés en intelligence artificielle et science des données. Les étudiants développeront des compétences clés telles que la conception et le développement de logiciels pour l'analyse automatique de grandes masses de données, la programmation du Big Data et de l'IA, la modélisation et l'automatisation du raisonnement sur les données, la conception d'architectures d'entreprise, l'application des méthodes statistiques, la maîtrise des techniques de gestion d'entreprise, l'utilisation de technologies pour l'exploitation de données réelles.

Ce parcours met l'accent sur le développement de solutions en géomatique en utilisant une expertise avancée dans la conception de systèmes d'information géographique, la télédétection et l'analyse spatiale. Les étudiants qui le suivent sont aptes à proposer des solutions informatiques adaptées aux besoins des utilisateurs en géomatique.

- **Mention « Relations Publiques et Multimédia », Parcours « Communication Multimédia (CM) »**

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Communication Multimédia (CM). Le Recrutement en S9 se fait par validation des crédits acquis.

Le secteur de la communication multimédia est un secteur extrêmement varié et concurrentiel. Cette formation s'appuie par exemple, sur la recherche et traitement d'information avant la mise en publication (dans un site web, radio, télévision).

Le double diplôme entre le parcours Cultures et Métiers du Web (CMW) de l'Université

Gustave Eiffel et le parcours Communication Multimédias (CM) de l'Université de Fianarantsoa propose l'obtention simultanée d'un diplôme de master en CMW et d'un diplôme de master en CM, avec deux suppléments de diplôme

- **Mention « Relations Publiques et Multimédia », Parcours « Communication Numérique et Management de Projet(CNMP*)* »**

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Communication Multimédia (CM). Le

Recrutement en S9 se fait par validation des crédits acquis.

La formation en Communication Numérique et Management de Projet de l'EMIT vise à former des professionnels polyvalents, compétents dans la gestion de projets numériques, le leadership d'équipes, et la compréhension des enjeux des médias numériques, en collaboration avec l'Université Bordeaux Montaigne en France.

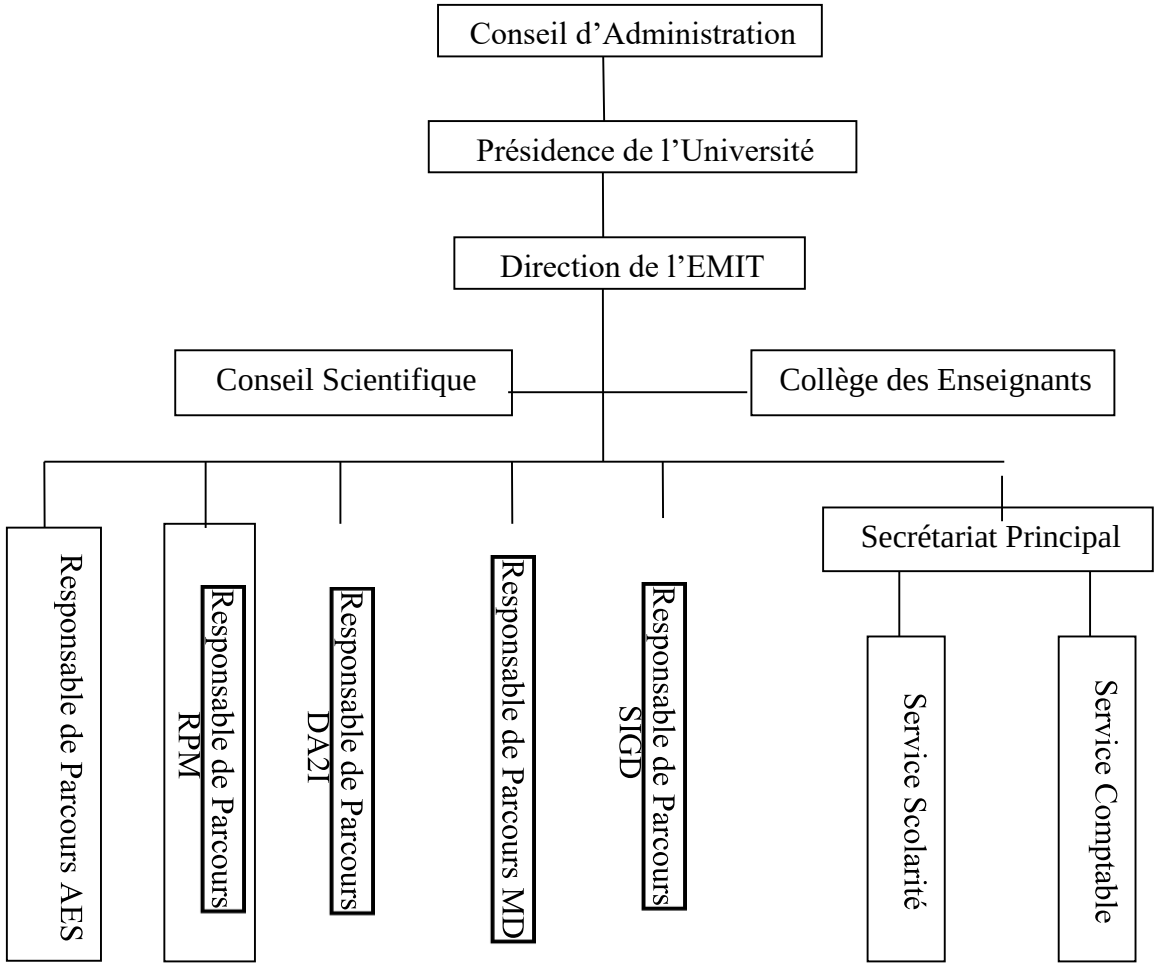
- **Mention « Relations Publiques et Multimédia », Parcours « Relations Publiques et Communication Organisationnelle (RPCO) »**

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Relations Publiques et Communication Organisationnelle (RPCO). Le Recrutement en S9 se fait par validation des crédits acquis.

Un parcours axé sur le développement de compétences stratégiques et opérationnelles. Les étudiants bénéficient d'une formation complète, couvrant la gestion de l'image de marque, la planification d'événements, la communication interne et externe, afin de les préparer à exceller dans le monde dynamique et exigeant des relations publiques.

1.4. Organisation

La structure hiérarchique au sein de l'EMIT se compose d'un conseil (scientifique ou d'établissement), une direction, un collège des enseignants, des chefs de mentions et des services présents au sein de l'école. Cette structure est représentée de manière générale sur la figure 1.1.



igure 1.1: Organigramme de l'EMIT

Source : Ecole de Management et d'Innovation Technologique

Chapitre 2 : Présentation ITDC Mada

2.1. Historique

La société a été créé afin d'attirer l'attention des étudiants sortant des écoles informatiques. Les objectifs de l'ITDC MADA s'articulent autour de plusieurs axes complémentaires qui reflètent sa mission et son engagement. Elle vise à accompagner numériquement les entreprises en leur proposant des solutions digitales adaptées, telles que des applications mobiles, web ou de bureau, afin d'améliorer leur performance et leur compétitivité. Elle vise aussi à promouvoir un développement durable et local en mobilisant les outils de la géomatique dans ses actions environnementales, urbaines, rurales et économiques. Par ailleurs, l'ITDC MADA accorde une grande importance à la collecte, au traitement et à l'analyse de données, qu'elles soient spatiales ou tabulaires, pour renforcer la prise de décision dans les projets de développement. Enfin, elle mise sur la synergie d'une équipe pluridisciplinaire composée de spécialistes issus de différents domaines, afin de garantir la réussite et la pertinence des actions menées.

2.2. Services

ITDC MADA propose une large gamme de services allant du :

2.2.1. Développement d'application web

ITDC MADA accompagne les entreprises dans la création de site web pour renforcer leur visibilité et valoriser leurs activités. La société conçoit également des applications web personnalisées afin d'optimiser les processus internes et faciliter la gestion de tâches professionnelles.

2.2.2. Développement d'applications de bureau

ITDC MADA propose un service de développement d'applications de bureau, conçues pour répondre aux besoins internes des entreprises. Ces applications facilitent la gestion quotidienne et automatisent les tâches pour améliorer l'efficacité opérationnelle.

2.2.3. Applications mobiles

L'équipe d'ITDC MADA dispose de l'expertise nécessaire pour concevoir et développer des applications mobiles, qu'elles soient natives ou hybrides. Des solutions sur mesure sont mises

en place afin de répondre aux besoins spécifiques des clients, tout en garantissant une expérience utilisateur fluide et optimale.

2.2.4. Géomatique

Dans le domaine du SIG, ITDC MADA est la première référence de la géomatique à Madagascar. Leur équipe de développeurs est à la hauteur d'effectuer une analyse thématique à partir des données spatiales, puis de l'intégrer aux cartes et aux données sur un site internet ou intranet avec webmapping.

2.3. Formations

Apprendre, évoluer, entreprendre. ITDC MADA offre des formations concrètes pour acquérir des compétences utiles et durables comme :

La création de site web

Développement d'application mobile

Initiation au marketing digital

Rédaction web

Initiation à la géomatique, le traitement de données SIG

Maîtrise des outils bureautiques et initiation à l'informatique

2.4. Mission

À travers ses services, ITDC MADA a pour mission de promouvoir le développement numérique et technologique en proposant des solutions innovantes en développement web, mobile et en géomatique pour répondre aux besoins de ses clients.

2.5. Clients de l'ITDC MADA

ITDC MADA accueille une grande diversité de clients, sans distinction de statut ou de taille. Elle collabore aussi bien avec des organismes gouvernementaux que non gouvernementaux, ainsi qu'avec des entreprises de toutes dimensions, qu'il s'agisse de petites et moyennes entreprises (PME), de grandes sociétés nationales ou d'entités étrangères.

Cette ouverture témoigne de sa capacité d'adaptation et de sa volonté de répondre à des besoins variés dans différents secteurs d'activités.

Grâce à une équipe constituée d'ingénieurs qualifiés et expérimentés, ITDC MADA est toujours prête à assurer la mise en œuvre de projets technologiques et numériques, en apportant des solutions concrètes, innovantes et adaptées aux exigences de ses partenaires.

Par ailleurs, l'entreprise ne se limite pas uniquement à la prestation de services techniques. Elle propose également des formations enrichissantes destinées au personnel administratif des entreprises ou des institutions. Ces sessions de formation ont pour objectif de renforcer les compétences des participants dans l'utilisation des outils informatiques et bureautiques, contribuant ainsi à une amélioration significative de leur productivité et de leur autonomie dans les tâches quotidiennes.

2.6. Méthode de travail

Chez ITDC MADA, la gestion des projets repose sur la méthode agile, qui permet d'assurer le bon déroulement de chaque mission en tenant compte du budget, des délais et surtout des besoins des clients. Cette approche vise à garantir la qualité, à maîtriser les coûts et à respecter les délais (QCD) tout en s'adaptant en permanence aux attentes des utilisateurs.

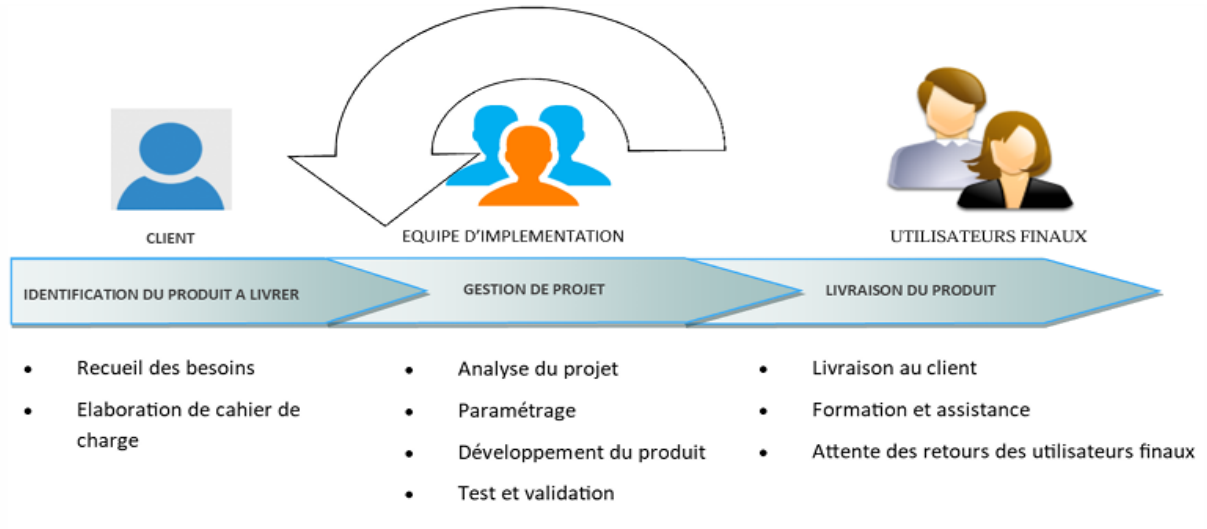


Figure 2.1: Méthode de travail d'ITDC Mada

2.7. Organigramme d'ITDC MADA

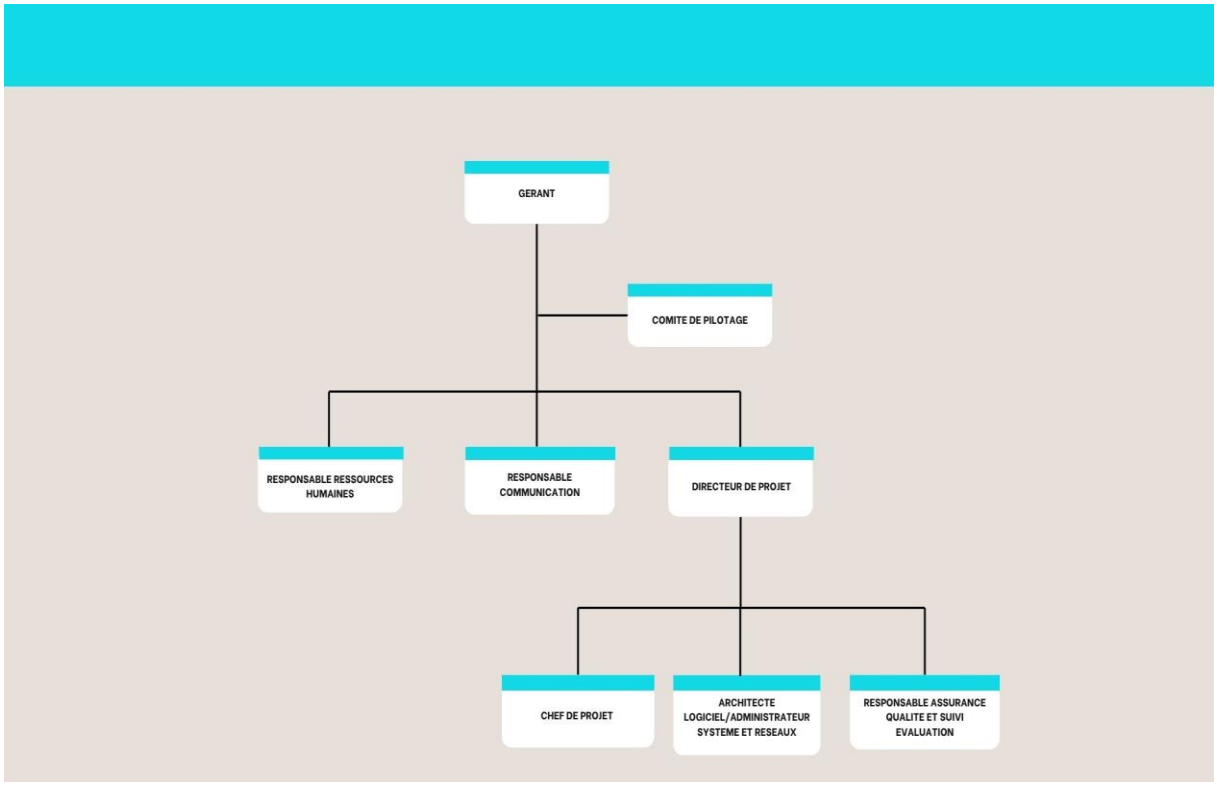


Figure 2.2 : Organigramme d'ITDC Mada

Chapitre 3 : Présentation du projet

Ce chapitre présente l'origine du projet, sa problématique et son importance, les objectifs et les besoins des utilisateurs, les moyens nécessaires et les résultats attendus.

3.1. Formulation du projet

3.1.1. Contexte institutionnel

L'Université de Fianarantsoa entretient des relations avec des entreprises, des institutions, des organismes et d'autres universités, notamment dans la formation, la recherche, l'innovation et la coopération institutionnelle. Les établissements doivent pouvoir enregistrer leurs propres partenaires, partenariats et projets dans une plateforme commune, tandis que l'équipe Projet et Partenariat de la Présidence doit disposer d'une vision globale de l'activité de coopération.

3.1.2. Situation avant le projet

Le suivi de ces activités était réalisé sous Excel. La plateforme a vocation à remplacer progressivement ce suivi Les limites générales d'un suivi sous tableur au regard des exigences sont présentées au tableau 4.1 (section 4.1.2)

3.1.3. Problématique

Le suivi sous Excel ne permet pas de répondre simultanément à plusieurs exigences : centraliser l'information, séparer strictement les données de chaque établissement, conserver l'historique des versions et archiver sans suppression définitive, calculer automatiquement les indicateurs et publier une partie de l'information à destination du public. La question centrale est celle de l'introduction : comment concevoir et réaliser une plateforme web qui permette de suivre ces projets et partenariats de manière centralisée, sécurisée et adaptée à chaque catégorie d'utilisateurs ?

3.2. Justification de l'importance du sujet

Le sujet répond à un besoin de pilotage institutionnel. Une université qui ne consolide pas ses projets et partenariats ne peut ni rendre compte de son activité, ni repérer les échéances de ses conventions, ni présenter une vision publique de sa coopération. Le sujet est aussi technique : il exige de concilier la centralisation des données avec une séparation stricte entre établissements, des rôles différents et un espace public, ce qui justifie une application web sur mesure.

3.3. Objectifs et besoins utilisateurs

3.3.1. Objectif général

Centraliser, gérer, suivre et archiver les informations relatives aux projets, partenariats, partenaires, cotutelles et conventions de l'Université, en remplaçant progressivement le suivi réalisé sous Excel.

3.3.2. Objectifs spécifiques

La plateforme doit permettre de :

gérer les partenaires et leurs contacts ;

gérer les partenariats entre établissements et partenaires ;

suivre les cotutelles de thèses ;

gérer les projets, leurs porteurs, bailleurs, bénéficiaires, sites et activités ;

suivre les activités des projets et, à terme, leur chronogramme ;

gérer les conventions et leurs documents ;

conserver l'historique des versions et archiver sans suppression définitive ;

générer une synthèse annuelle par établissement ;

gérer les publications destinées à l'espace public ;

notifier les utilisateurs des ajouts, échéances, expirations et retards.

3.3.3. Acteurs et droits d'accès

Le tableau 3.1 présente les acteurs et leurs droits. L'authentification se fait par identifiant et mot de passe, et le rôle du compte détermine l'espace accessible et les autorisations. La création des comptes relève du périmètre de la Présidence

Tableau 3.1 : Acteurs et droits d'accès

Acteur	Droits
Équipe Projet et Partenariat (Présidence)	Vision globale ; gestion complète des partenaires, partenariats, cotutelles, projets, conventions, documents, archives, publications et notifications ; production et complétude de la synthèse annuelle.
Établissement	Compte unique ; consultation, ajout et modification de ses propres données ; aucun accès aux données d'un autre établissement ; les informations créées sont rattachées automatiquement à l'établissement du compte.
Décideur	Tableau de bord global, indicateurs agrégés,

Acteur	Droits
	lecture seule du détail des fiches ; aucune opération de gestion.
Visiteur	Sans compte ; consultation des informations et publications rendues publiques.

Source : cahier des charges consolidé, section 2.

3.3.4. Besoins fonctionnels

Le tableau 3.2 présente les besoins fonctionnels principaux, par module.

Tableau 3.2 : Besoins fonctionnels principaux

Module	Besoins
Partenaires et contacts	Fiche (nom, logo, type, pays, coordonnées, site web, statut, visibilité publique ou privée, indicateur « partenaire stratégique »), contacts multiples, recherche préalable pour limiter les doublons.
Partenariats	Relation entre un ou plusieurs établissements et un ou plusieurs partenaires ; regroupe plusieurs projets ; champs de suivi (actions réalisées, en cours, prévues, résultats, difficultés, perspectives).
Cotutelles de thèses	Collaboration académique liée à un doctorant et à une université partenaire ; directeurs, dates, financement, mobilité, publications, avancement.
Projets	Nom et intitulé officiel distincts, type, budget, dates, statut, établissement responsable, porteur, bailleurs, partenaires directs, bénéficiaires, sites, activités ; détection des projets en retard.
Activités et chronogramme	Avancement (%) = activités terminées / total des activités × 100, indépendant du chronogramme (non implémenté dans cette version). Chronogramme (année, trimestre, mois, jours consacrés), indépendant de l'avancement.
Documents et archivage	TDR, descriptions, conventions ; ancienne version conservée lors d'un remplacement ; archivage sans suppression définitive.
Conventions	Référence, titre, dates, statut, documents ; archivage et téléchargement des conventions signées ; surveillance des dates de validité.
Publications	Entité distincte du projet ; statut Brouillon ou Publiée ; photos avec légende et ordre ; seules les publications publiées sont visibles du public.
Recherche et tableaux de bord	Recherche et filtres ; tableau de bord par rôle (Présidence, établissement, Décideur).
Synthèse annuelle	Calculée automatiquement par établissement ; éléments narratifs saisis manuellement (domaines principaux, partenaires stratégiques, défis, priorités).
Notifications	Ajouts, expirations de conventions (seuils de 30, 15 et 7 jours), activités en retard, projets dépassant leur date de fin ; état lu ou non lu ; contrôles périodiques.
Espace public	Accueil, À propos, Établissements, Partenaires, Projets et Partenariats, Contact.

Source : cahier des charges consolidé, section 3.

3.3.5. Règle de rattachement des conventions

Une convention est une entité indépendante. Elle peut être liée à un partenariat, à un projet, à une cotutelle, à plusieurs de ces entités à la fois, ou à aucune. Les trois liens sont facultatifs et indépendants. La règle précédente, qui imposait au moins un lien, a été supprimée : elle contredisait les champs optionnels et empêchait les conventions-cadres ou autonomes. Le circuit de traitement interne (Projet et Partenariat, Secrétaire principale, Direction des Affaires Juridiques, signature) reste suivi manuellement.

3.3.6. Besoins non fonctionnels

Le cahier des charges fixe les exigences suivantes : simplicité d'utilisation pour les établissements ; séparation stricte des données entre établissements ; accès contrôlé aux documents et aux archives, avec stockage sécurisé des fichiers et des photos ; performances acceptables lorsque le volume augmente ; architecture maintenable et évolutive ; notifications fiables, périodiques et non répétitives à l'excès ; identité visuelle institutionnelle (palette verte #2F7D5D et orange #F2B233, typographie Inter, rayons de bordure de 6 à 12 px).

3.4. Moyens nécessaires à la réalisation du projet

3.4.1. Moyens humains

Le projet a été réalisé en binôme. Le tableau 3.3 présente la répartition des responsabilités. Le dépôt compte trois contributeurs humains : l'auteur (irielamatthieu, 181 commits), le binôme backend (Yastellin, 219 commits) et un troisième contributeur (iraotomiarina, 43 commits), ainsi que des comptes d'agent utilisés pour les tâches automatisées

Tableau 3.3 : Répartition des responsabilités au sein du binôme

Domaine	**Responsable**	**Périmètre**
Frontend	L'auteur du présent mémoire (RAOTOMIARINA Iriela)	Interface utilisateur, ergonomie, intégration des maquettes, consommation de l'API, expérience utilisateur, tests d'interface.
Backend	TSIVAHINY Yastellin Barbis	Base de données, API, logique métier, sécurité, authentification.

3.4.2. Moyens matériels

Le développement a été réalisé sur un poste Windows 10 Core i5 5 ème géneration , 8 Go de RAM, l'environnement de développement étant exécuté par Docker (voir section 8.2).,

3.4.3. Moyens logiciels

Les moyens logiciels sont décrits au chapitre 4 (choix des outils) et au chapitre 7 (configuration). Ils comprennent Git pour le versionnement, Docker et Docker Compose pour l'environnement reproductible, Swagger pour la documentation de l'API, draw.io pour les diagrammes et un fichier de suivi de la dette technique.

3.5. Chronogramme de réalisation du projet

Le projet a suivi une approche Agile, par itérations courtes. Le dépôt Git s'étend du 31 août 2026 au 2 novembre 2026 et compte 455 commits sur la branche develop au 6 novembre 2026. Le chronogramme détaillé par itération n'est pas encore établi dans ce document

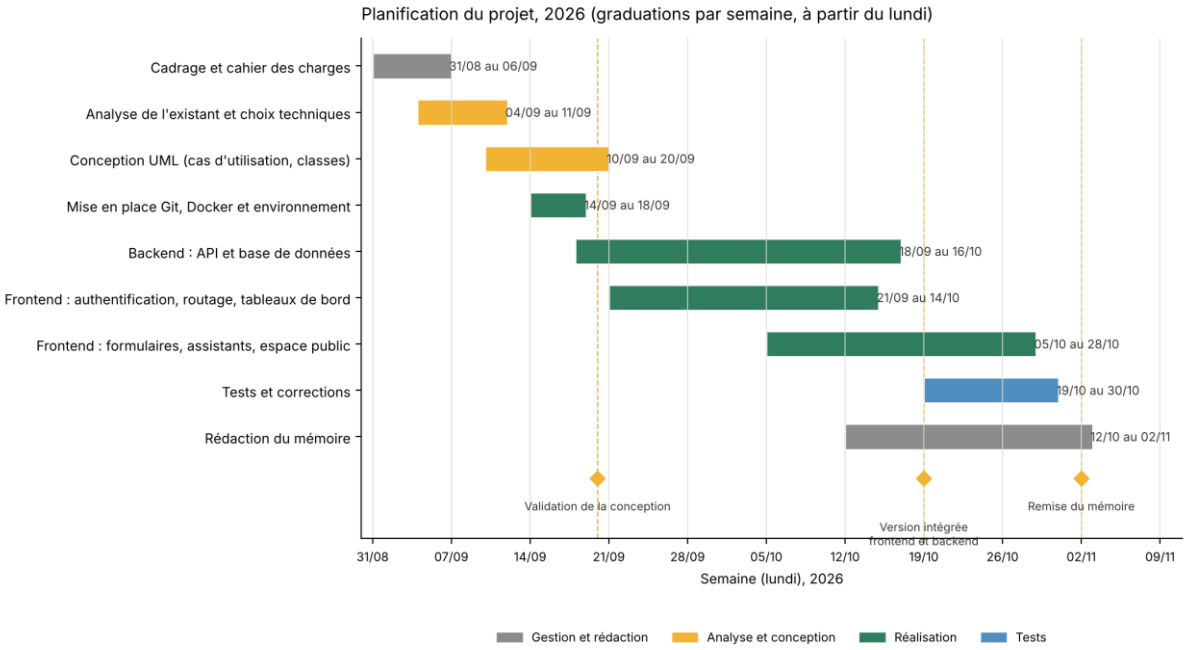


Figure 3.1 : Planification du projet (diagramme de Gantt)

3.6. Résultat attendu

Le résultat attendu est une plateforme web qui centralise la gestion des partenaires, partenariats, cotutelles, projets, conventions et publications, sépare les données par établissement, assure l'archivage et l'historique, produit une synthèse annuelle et notifie les échéances. Les résultats effectivement constatés sont confrontés à ces attentes au chapitre 9.

PARTIE 2 : ANALYSE ET CONCEPTION

Chapitre 4 : Analyse préalable

Ce chapitre analyse l'existant, justifie la solution retenue et les choix méthodologiques, techniques et d'outils.

4.1. Analyse de l'existant

4.1.1. Organisation actuelle

Le suivi des projets, partenariats et conventions était réalisé sous Excel. Les établissements transmettaient leurs informations à l'équipe Projet et Partenariat de la Présidence, qui les consolidait. La plateforme a pour objectif de remplacer progressivement ce fonctionnement

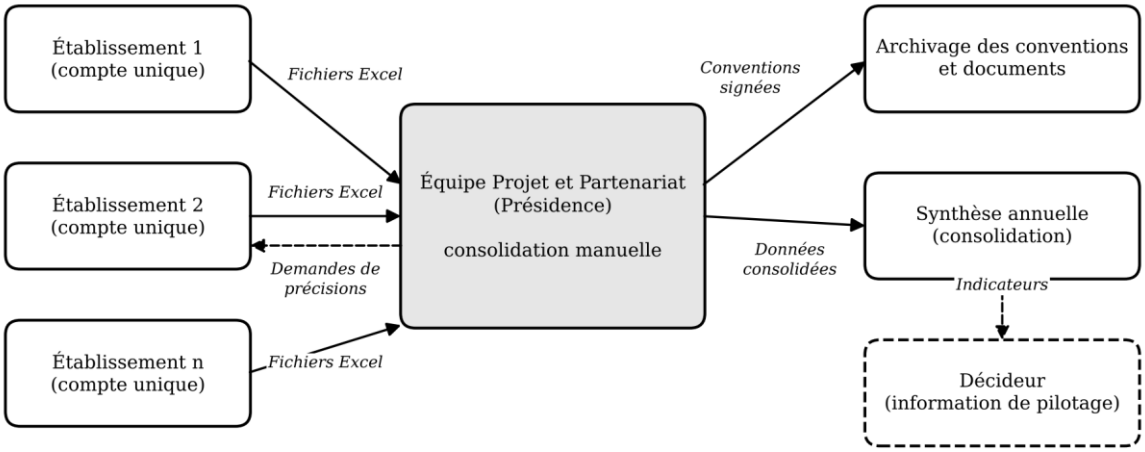


Figure 4.1 : Diagramme de flux de l'organisation actuelle

4.1.2. Critique de l'existant

Le tableau 4.1 confronte les caractéristiques d'un suivi sous tableur aux exigences du cahier des charges.

Tableau 4.1 : Limites du suivi sous Excel au regard des exigences

Exigence du cahier des charges	Limite d'un suivi sous tableur
Centraliser les informations des établissements	Données réparties dans plusieurs fichiers, à consolider manuellement
Séparer strictement les données de chaque établissement	Pas de contrôle d'accès par rôle au sein d'un même fichier
Conserver l'historique et archiver sans suppression définitive	Versions multiples difficiles à tracer
Calculer automatiquement l'avancement et les	Calculs manuels ou formules à maintenir

Exigence du cahier des charges	Limite d'un suivi sous tableur
synthèses	
Notifier les échéances, expirations et retards	Aucune alerte automatique
Gérer les relations entre entités	Relations difficiles à représenter et à contrôler
Publier une partie de l'information au public	Aucun espace public
Associer des documents et des photos	Pièces jointes gérées à part

Source : analyse de l'auteur à partir du cahier des charges.

4.1.3. Moyens existants

Les moyens actuels sont humains (les établissements et l'équipe Projet et Partenariat) et logiciels (le tableur).

4.2. Choix de la solution

Trois solutions ont été envisagées. La solution A améliore le suivi sous Excel : elle est rapide à mettre en place, mais ne résout ni le contrôle d'accès, ni les notifications, ni l'espace public. La solution B adopte un logiciel existant : elle offre des fonctionnalités éprouvées, mais risque de ne pas correspondre aux règles propres à l'Université (rôles, cotutelles, conventions à rattachements facultatifs, synthèse annuelle) et peut impliquer des coûts de licence. La solution C développe une plateforme web sur mesure : elle répond précisément aux besoins, mais exige un effort de développement et de maintenance.

Tableau 4.2 : Comparaison des solutions envisagées

Critère	A : Excel amélioré	B : Logiciel existant	C : Plateforme sur mesure
Adaptation aux règles métier	Faible	Moyenne	Forte
Contrôle d'accès par rôle	Non	Variable	Oui
Notifications automatiques	Non	Variable	Oui
Espace public intégré	Non	Variable selon le logiciel	Oui
Coût initial	Faible	Licences possibles	Développement
Maîtrise de l'évolution	Faible	Limitée	Totale

Source : analyse de l'auteur.

La solution C a été retenue : c'est la seule qui couvre l'ensemble des exigences, soit trois espaces de gestion par rôle et un espace public, la séparation stricte des données, la traçabilité par historique et archives, des notifications périodiques, une synthèse annuelle calculée automatiquement et une identité visuelle institutionnelle. La plateforme est une application à trois niveaux : un frontend monopage (SPA) en React et TypeScript, un backend exposant une API REST avec NestJS, et une base PostgreSQL.

4.2.1. Faisabilité technique

Le projet repose sur des technologies libres et bien documentées : React, TypeScript, NestJS, TypeORM et PostgreSQL. L'environnement conteneurisé avec Docker garantit une installation reproductible

4.2.2. Faisabilité organisationnelle

Le travail est réparti entre deux personnes aux périmètres clairs, reliées par un contrat d'API. Les évolutions du cahier des charges ont été intégrées au fil du développement grâce à la démarche Agile

4.2.3. Faisabilité temporelle

Le stage dure trois mois. Le périmètre étant large, il a été traité par modules livrés progressivement. Le modèle et la migration du chronogramme existent dans le backend, mais son interface n'est pas disponible dans cette version. Les modules livrés sont ceux du tableau 9.1 ; l'interface du chronogramme et le calcul de l'avancement en pourcentage n'ont pas été livrés (section 9.3.1)..

4.2.4. Faisabilité économique

Les outils utilisés sont gratuits ou libres, ce qui limite le coût au temps de développement.

4.2.5. Conclusion sur la faisabilité

Au regard de ces quatre critères, le projet est jugé réalisable dans le cadre du stage

4.3. Choix de la méthode de conception et de gestion de projet

4.3.1. Approche Agile

Les méthodes Agiles reposent sur le Manifeste pour le développement logiciel Agile publié en 2001 (Beck et al., 2001). Elles privilégient les individus et leurs interactions, un logiciel opérationnel, la collaboration avec le client et l'adaptation au changement. Le développement s'organise en itérations courtes, chacune produisant un incrément présenté à l'encadrant, testé puis amélioré. Cette approche a été retenue pour quatre raisons : le périmètre large se livre par incréments ; les besoins ont évolué, comme la règle de rattachement des conventions, supprimée car elle empêchait les conventions-cadres ; le frontend et le backend progressent en parallèle autour d'un contrat d'API ; la durée de trois mois impose de livrer rapidement des résultats visibles.

Pendant le développement, la démarche s'est traduite par des cycles courts, versionnés avec Git ; une intégration progressive du frontend et du backend ; des ajustements successifs du cahier des charges ; un suivi de la dette technique dans un fichier dédié ; des tests automatisés exécutés au fil des évolutions.

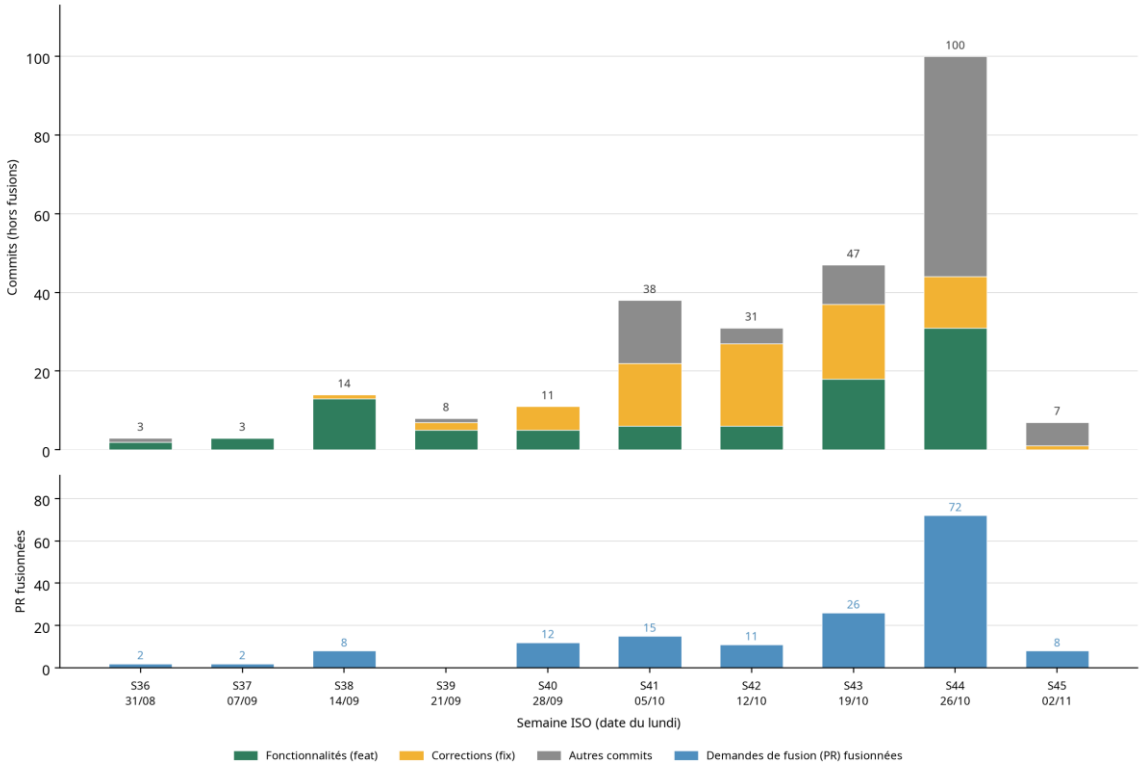


Figure 4.2 : Place de l'approche Agile dans le déroulement du projet

4.3.2. Notation de modélisation : le langage UML

UML (Unified Modeling Language) est un langage graphique normalisé de modélisation des systèmes logiciels, issu de la fusion, dans les années 1990, des travaux de Grady Booch, Ivar Jacobson et James Rumbaugh, puis adopté comme standard par l'Object Management Group (OMG) (Booch et al., 1999). UML n'est pas une méthode : il fournit une notation indépendante de la démarche, qui se marie avec une approche Agile.

UML a été retenu pour quatre raisons : c'est un standard reconnu, connu des développeurs et de l'encadrant ; il est cohérent avec l'approche orientée objet de la pile technique, car les entités du backend et les composants du frontend se transposent en classes ; il couvre les fonctionnalités, la structure des données, les échanges, les processus et les cycles de vie ; il peut être réalisé de manière légère et mis à jour à chaque itération.

4.4. Choix de l'outil de conception

Le projet s'appuie sur un sous-ensemble de diagrammes UML réalisés avec draw.io, outil gratuit qui permet de les produire et de les exporter en image. Le tableau 4.3 présente les diagrammes retenus.

Tableau 4.3 : Diagrammes UML retenus pour le projet

Diagramme	Objectif	Application au projet	Chapitre
Cas d'utilisation	Décrire les fonctionnalités du point de vue des acteurs	Équipe Projet et Partenariat, Établissement, Décideur, Visiteur	5
Classes	Décrire la structure des données et leurs relations	Entités du cahier des charges	5 et 6
Séquence	Décrire les échanges entre composants	Authentification, création d'une convention	5 et 6
Activités	Décrire un processus	Circuit de traitement d'une convention	6
États-transitions	Décrire le cycle de vie d'un objet	Statuts d'une convention, d'un projet, d'une publication	6

Source : analyse de l'auteur.

Pour assurer la cohérence des modèles et du code, plusieurs conventions ont été retenues : les identifiants métier (colonnes, champs, tables) sont en français, sans accents, en snake_case, conformément au cahier des charges ; les classes suivent la convention PascalCase ; les relations précisent leur multiplicité ; les rattachements facultatifs sont représentés par une multiplicité 0..1 ; les tables d'association, comme ProjetPartenaire, traduisent les relations plusieurs-à-plusieurs ; les statuts sont modélisés par des énumérations ; les figures sont numérotées par chapitre avec une légende placée dessous, et les tableaux avec une légende placée au-dessus.

4.5. Choix du langage de programmation

4.5.1. TypeScript

4.5.1.1. Frontend

Le frontend est écrit en TypeScript, version 6.0. Le typage statique décrit les formes des données échangées avec l'API, ce qui réduit les erreurs d'intégration.

4.5.1.2. Backend

Le backend est écrit en TypeScript, version 5.7. Le même langage facilite la maintenance d'un code partagé par deux personnes et la lecture des contrats d'API.

4.6. Choix du framework

4.6.1. Framework frontend

Pour le frontend, trois options étaient envisageables : React, Vue et Angular. Le tableau 4.4 les compare. React, dans sa version 19, a été retenu pour sa grande liberté d'architecture, son écosystème de composants et la possibilité d'organiser le code par fonctionnalités.

Tableau 4.4 : Comparaison des options pour le frontend

Critère	React	Vue	Angular
Nature	Bibliothèque d'interface	Framework progressif	Framework complet
Typage	TypeScript largement adopté (fichiers TSX)	TypeScript possible	TypeScript natif
Organisation du code	Libre, à structurer par le développeur	Conventions légères	Conventions strictes imposées
Écosystème de composants	Très riche (dont shadcn/ui, Radix UI)	Riche	Riche, souvent lié au framework
Adaptation au projet	Forte : composants réutilisables, grande liberté	Bonne	Bonne, mais plus lourde pour un binôme

Source : analyse de l'auteur, à partir de la documentation officielle de chaque outil.

4.6.2. Framework backend

Pour le backend, l'API REST est développée avec NestJS 11, organisée en modules fonctionnels et documentée avec Swagger.

4.7. Choix de l'environnement de développement

L'environnement repose sur Git et GitHub pour le versionnement et le partage entre les deux contributeurs, sur Docker et Docker Compose pour un environnement reproductible (frontend, backend et base de données), sur Nginx 1.27 pour servir les fichiers statiques du frontend en production, sur draw.io pour les diagrammes et sur Swagger pour tester les routes de l'API.

4.7.1. Outils du frontend

Le tableau 4.5 liste les bibliothèques du frontend, avec leur version et leur rôle.

Tableau 4.5 : Outils et bibliothèques du frontend

Outil	Version	Rôle dans le projet

Outil	Version	Rôle dans le projet
React	19.2	Construction de l'interface par composants
TypeScript	6.0	Typage statique du code
Vite	8.2	Serveur de développement et compilation
Tailwind CSS	4.3	Mise en forme par classes utilitaires
shadcn/ui et Radix UI	4.21 et 1.6	Composants d'interface accessibles (boîtes de dialogue, onglets, sélecteurs)
React Router	7.18	Routage côté client
TanStack Query	5.102	Gestion de l'état serveur et mise en cache des requêtes
React Hook Form et Zod	7.87 et 4.5	Gestion et validation des formulaires
Axios	1.20	Client HTTP vers l'API
Recharts	3.10	Graphiques des tableaux de bord et de la synthèse
Framer Motion	13.2	Animations et transitions de pages
Sonner	2.0	Messages de notification à l'écran
next-themes	0.4	Thème clair et thème sombre
react-dropzone	20.1	Dépôt de fichiers par glisser-déposer
Inter Variable (@fontsource-variable/inter)	5.3	Police de l'interface
Vitest et jsdom	5.0 et 30.1	Tests unitaires du frontend
oxlint	1.79	Analyse statique du code (lint)

Source : fichier package.json du frontend.

4.7.2. Outils du backend

Le tableau 4.6 liste les bibliothèques du backend, avec leur version et leur rôle.

Tableau 4.6 : Outils du backend

Outil	Version	Rôle dans le projet
NestJS	11	Framework de l'API REST, organisé en modules
TypeORM	0.3.31	Mappage objet-relationnel (ORM) et migrations
@nestjs/passport et passport-jwt	11 et 4.0.1	Authentification par jetons
bcrypt	5.1	Hachage des mots de passe
Helmet et Throttler	8.3 et 6	En-têtes de sécurité HTTP et limitation du débit des requêtes
class-validator et class-transformer	0.14 et 0.5	Validation des objets de transfert de données (DTO)
Swagger	11	Documentation interactive de l'API
Multer	1.4	Téléversement de fichiers
ExcelJS, PDFKit, Archiver	4.4, 0.15, 5.3	Exports XLSX et PDF, et archives
Nodemailer	10	Envoi de courriels via SMTP
@nestjs/schedule	5	Tâches planifiées (notifications, nettoyage)
Jest, Supertest, PGlite	29, 7, 0.5	Tests unitaires et de bout en bout

Outil	Version	Rôle dans le projet
		(E2E) sur une base légère

Source : fichier package.json du backend.

4.8. Choix du système de gestion de base de données

La base de données est PostgreSQL 16, système relationnel interrogé en SQL. Le modèle comporte des relations plusieurs-à-plusieurs passant par des tables d'association, des rattachements facultatifs traduits par des clés étrangères nullables, et des identifiants de type UUID. Ces caractéristiques s'accordent avec le modèle de données et avec l'ORM TypeORM.

Chapitre 5 : Conception du projet

5.1. Présentation de la méthode de conception

La conception s'appuie sur les diagrammes UML du chapitre 4, selon trois points de vue : fonctionnel (cas d'utilisation), statique (classes et relations) et dynamique (séquences, activités, états). Elle suit une démarche itérative, cohérente avec l'approche Agile retenue.

5.2. Dictionnaire des données

Le modèle de données regroupe les classes présentées au tableau 5.1. Les énumérations du modèle sont présentées au tableau 5.2. Le backend compte 33 fichiers d'entités (*.entity.ts) ; la figure 5.3 présente 19 classes.

Tableau 5.1 : Classes du modèle par domaine

Domaine	Classes principales
Comptes et sécurité	Utilisateur, Établissement, jetons de rafraîchissement, jetons de réinitialisation de mot de passe
Partenaires	Partenaire, Contact, Coordonnée
Partenariats et cotutelles	Partenariat, PartenariatÉtablissement, PartenariatPartenaire, CotutelleThèse
Projets	Projet, Activité, Chronogramme, Porteur, Bailleur, Bénéficiaire, Site, et leurs tables d'association (ProjetPartenaire, ProjetBailleur, ProjetBénéficiaire, ProjetSite)
Conventions, documents et publications	Convention, Document, Publication, PhotoPublication
Suivi et pilotage	Notification, HistoriqueAction, SyntheseItem, SynthèsePartenaireStratégique, MessageContact, ExportHistorique, Configuration

Source : cahier des charges consolidé, section 5, et analyse du code du backend.

Tableau 5.2 : Énumérations du modèle

Énumération	Valeurs
Rôle	PRESIDENCE, ETABLISSEMENT, DECIDEUR
Statut du projet	BROUILLON, EN_ATTENTE, EN_COURS, SUSPENDU, EXPIRE, TERMINE
Statut du partenariat	BROUILLON, EN_ATTENTE, EN_COURS, SUSPENDU, TERMINE
Statut de la convention	BROUILLON, EN_ATTENTE_SIGNATURE, SIGNEE, ACTIVE, SUSPENDUE, EXPIREE
Statut du partenaire	ACTIF, SUSPENDU
Statut de la publication	BROUILLON, PUBLIEE

Source : code du backend (dossier backend/src/common/enums).

5.3. Règles de gestion

Le cahier des charges consolidé présente les règles de gestion principales. Les plus structurantes sont les suivantes : un compte unique pour la Présidence et un compte unique par établissement, un établissement ne gérant que ses propres données ; un ou plusieurs comptes Décideur sans rattachement, en lecture seule ; un projet peut avoir plusieurs activités et être rattaché à au plus un partenariat, ou à aucun ; un partenariat peut concerner plusieurs établissements et partenaires, et regrouper plusieurs projets ; l'avancement dépend des activités terminées et reste indépendant du chronogramme ; l'archivage conserve les données et les anciennes versions ; seules les publications publiées sont visibles publiquement ; les contrôles d'échéance et de retard sont périodiques.

La documentation de migration du dépôt (fichier MIGRATION_CONTEXT.md) cite les règles RG01 à RG44 en renvoyant au cahier des charges externe. Dans le dépôt, la règle RG01 et la règle RG44 apparaissent dans cette documentation, et la règle RG28 apparaît dans un commentaire du formulaire de contact des partenaires. Le reste du code ne cite pas les autres règles.

5.4. Diagramme de cas d'utilisation

Quatre acteurs interagissent avec le système : l'équipe Projet et Partenariat (Présidence), l'établissement, le Décideur et le visiteur. Les trois premiers utilisent le cas commun « S'authentifier ». Le tableau 5.3 décrit les besoins fonctionnels par acteur, qui forment les cas d'utilisation représentés à la figure 5.1.

Tableau 5.3 : Besoins fonctionnels par acteur

Acteur	Fonctionnalités attendues
Équipe Projet et Partenariat (Présidence)	S'authentifier ; consulter un tableau de bord global ; gérer les partenaires, partenariats, cotutelles, projets, conventions et documents ; gérer les publications ; consulter les archives ; recevoir les notifications ; produire et compléter la synthèse annuelle.
Établissement	S'authentifier ; consulter un tableau de bord limité à ses données ; rechercher ou créer un partenaire ; créer et modifier ses partenariats et projets ; suivre les activités et le chronogramme ; gérer ses conventions, documents et publications ; recevoir les notifications.
Décideur	S'authentifier ; consulter un tableau de bord global et des indicateurs agrégés ; consulter le détail des fiches en lecture seule.
Visiteur	Consulter l'accueil, la page À propos, les établissements, les partenaires, les projets et partenariats publiés ; envoyer un message via la

Acteur	Fonctionnalités attendues
	page Contact.

Source : cahier des charges consolidé, sections 2, 3 et 10.

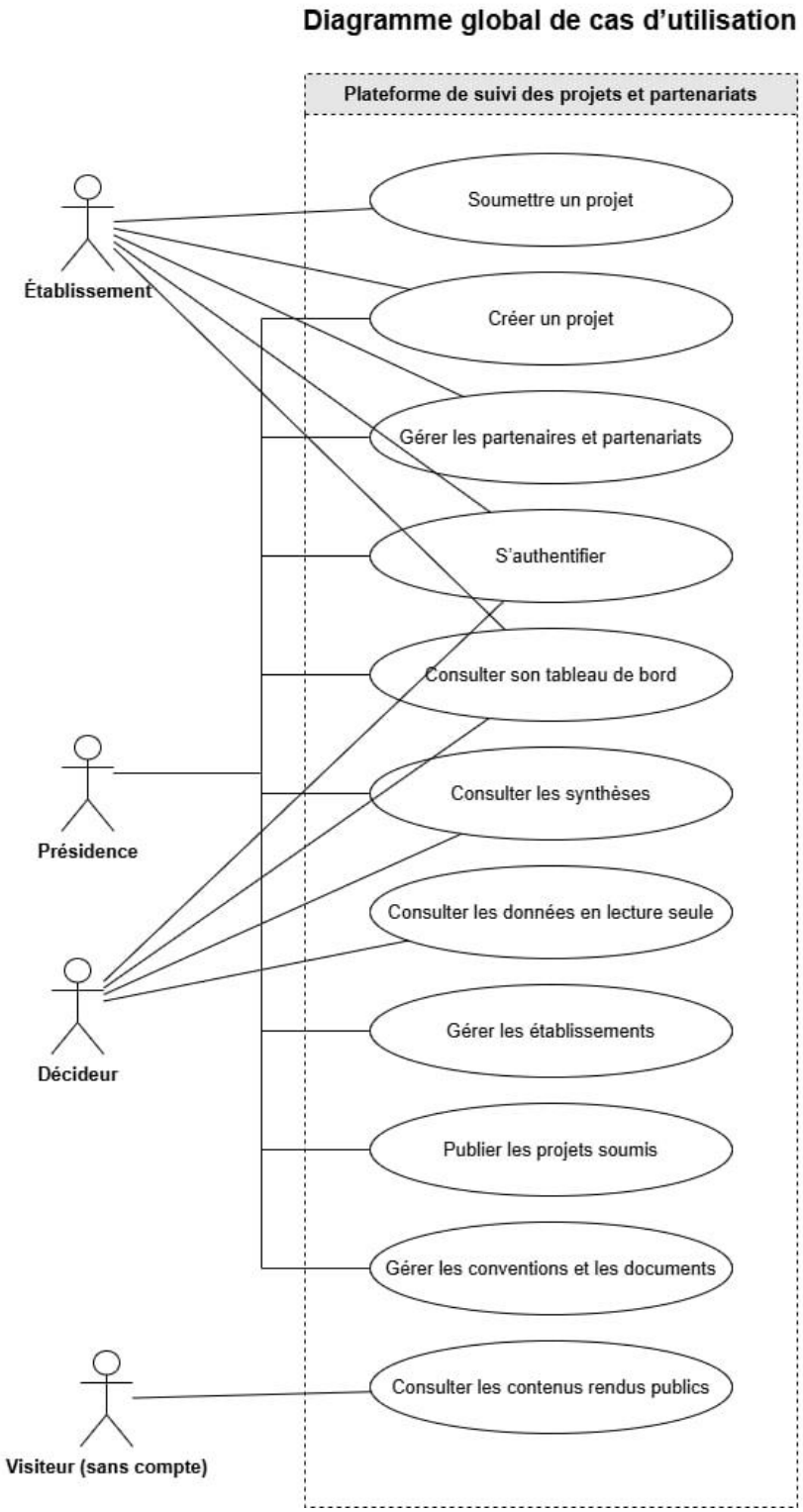


Figure 5.1 : Diagramme de cas d'utilisation général

Plusieurs besoins transversaux concernent l'ensemble de la plateforme : recherche et filtres ; calcul de l'avancement d'un projet (nombre d'activités terminées divisé par le nombre total d'activités, indépendant du chronogramme, fonctionnalité non implémentée dans cette version) ; archivage sans suppression définitive ; notifications automatiques ; synthèse annuelle calculée par établissement ; séparation des informations publiques et internes.

5.5. Description textuelle des cas d'utilisation

5.5.1. Cas « Créer une convention »

Tableau 5.4 : Cas d'utilisation « Créer une convention »

Rubrique	Description
Acteurs	Équipe Projet et Partenariat, établissement
Précondition	L'utilisateur est authentifié.
Scénario nominal	1. L'utilisateur ouvre le formulaire de convention. 2. Il saisit le titre, le statut et les dates. 3. Il sélectionne éventuellement un partenariat, un projet ou une cotutelle. 4. Il valide. 5. Le système enregistre la convention.
Scénario alternatif	Si la date de fin précède la date de début, le formulaire affiche une erreur et refuse l'envoi.
Postcondition	La convention est créée avec zéro, un, deux ou trois rattachements.

Source : cahier des charges consolidé, sections 3 et 8.

5.5.2. Cas « S'authentifier »

Tableau 5.5 : Cas d'utilisation « S'authentifier »

Rubrique	Description
Acteurs	Équipe Projet et Partenariat, établissement, Décideur
Précondition	L'utilisateur possède un compte.
Scénario nominal	1. L'utilisateur saisit son identifiant et son mot de passe. 2. Le système vérifie les informations. 3. Le système délivre un jeton d'accès et un jeton de rafraîchissement. 4. L'utilisateur est dirigé vers l'espace correspondant à son rôle.
Scénario alternatif	Si les informations sont incorrectes, un message d'erreur s'affiche.
Postcondition	L'utilisateur est connecté et ses droits sont déterminés par son rôle.

Source : cahier des charges consolidé, section 2.5.

5.6. Diagramme de séquence système

Le diagramme de séquence système décrit les échanges entre l'acteur et le système, sans détailler les composants internes. Pour le cas « S'authentifier », l'utilisateur saisit ses identifiants, le système vérifie les informations et délivre les jetons, puis l'utilisateur accède à

son espace. Pour le cas « Créer une convention », l'utilisateur remplit le formulaire, le système valide les données, enregistre la convention et confirme l'opération.

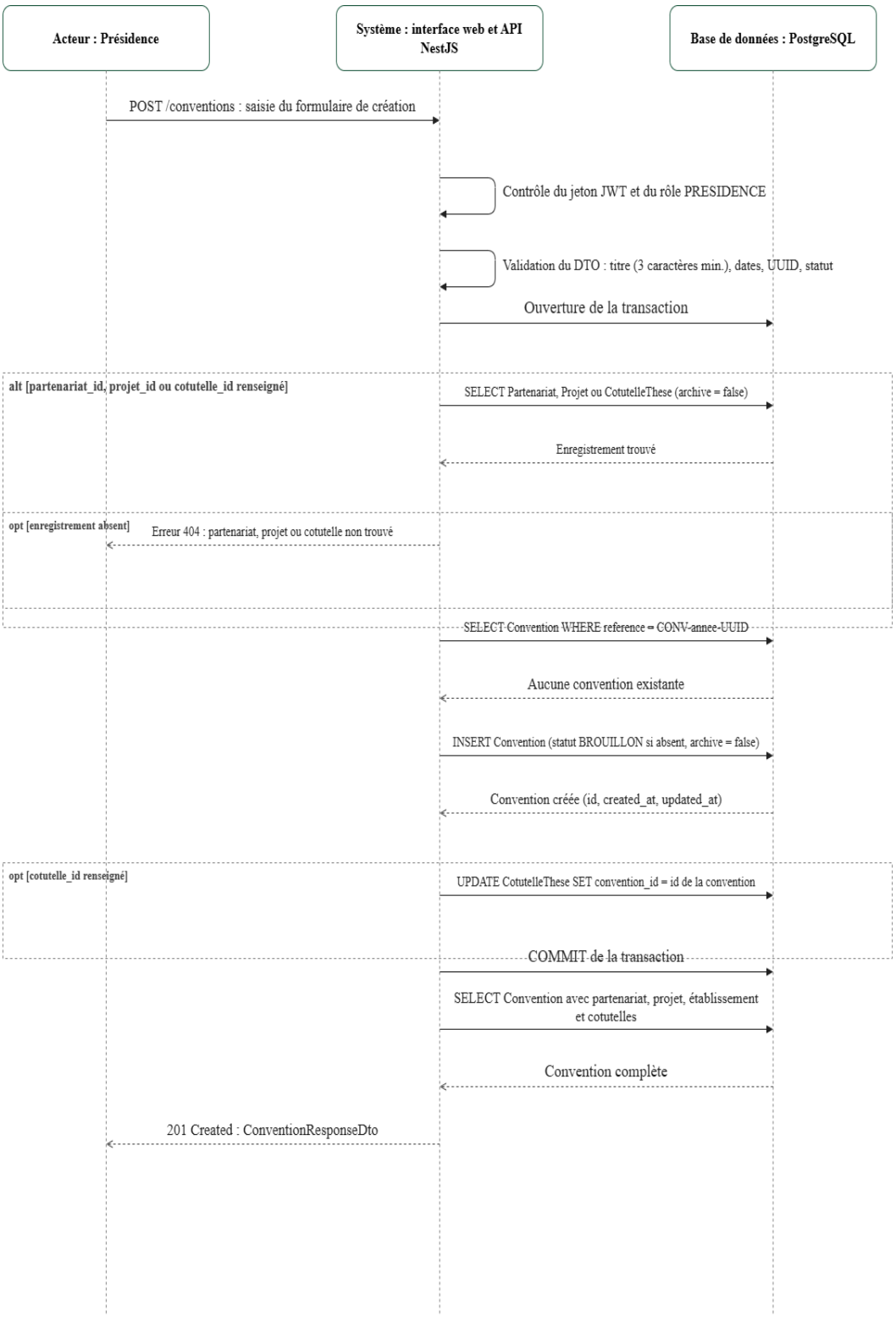


Figure 5.2 : Diagramme de séquence système du cas « Créer une convention »

5.7. Modèle de domaine

Le modèle de domaine rassemble les entités et leurs relations principales. Le tableau 5.6 présente les multiplicités retenues. Les rattachements facultatifs sont représentés par une multiplicité 0..1.

Tableau 5.6 : Relations principales du modèle

Association	Multiplicité côté A	Multiplicité côté B	Remarque
Établissement (A) – Utilisateur (B)	0..1	0..*	Présidence et Décideur sans rattachement
Établissement (A) – Projet (B)	0..1	0..*	Colonne projet.etablissement_id nullable : un projet créé par la Présidence peut n'avoir aucun établissement
Partenariat (A) – Projet (B)	0..1	0..*	Vérifié
Partenariat (A) – Établissement (B)	0..*	1..*	Via PartenariatÉtablissement
Partenariat (A) – Partenaire (B)	0..*	1..*	Via PartenariatPartenaire
Projet (A) – Partenaire direct (B)	0..*	0..*	Via ProjetPartenaire
Activité (A) – Chronogramme (B)	1	0..*	Plusieurs entrées de chronogramme par activité
Convention (A) – Partenariat (B)	0..*	0..1	Vérifié : rattachement facultatif
Convention (A) – Projet (B)	0..*	0..1	Vérifié : rattachement facultatif
Convention (A) – Cotutelle (B)	0..1	0..*	La clé est portée par la cotutelle (voir la remarque ci-dessous)
Convention (A) – Document (B)	0..1	0..*	Un document peut aussi être lié à un projet, une cotutelle ou un partenariat
Publication (A) – Photo (B)	1	0..*	Photos avec légende et ordre d'affichage

Source : cahier des charges consolidé, sections 3, 4 et 5, et analyse du code. La multiplicité indiquée correspond au nombre d'instances de la classe B associées à une instance de la classe A.

Remarque sur la convention. Le cahier des charges définit les clés partenariat_id, projet_id et cotutelle_id comme facultatives sur l'entité Convention. L'analyse du code montre que partenariat_id et projet_id sont des colonnes nullables de la convention, mais que la clé étrangère de la cotutelle est portée par l'entité CotutelleThèse, qui référence une convention. Une convention peut ainsi être liée à plusieurs cotutelles dans le code, alors que le cahier des

charges prévoit une seule valeur. Deux solutions sont possibles : ajouter une colonne cotutelle_id nullable à la convention, avec une migration, ou corriger le cahier des charges.

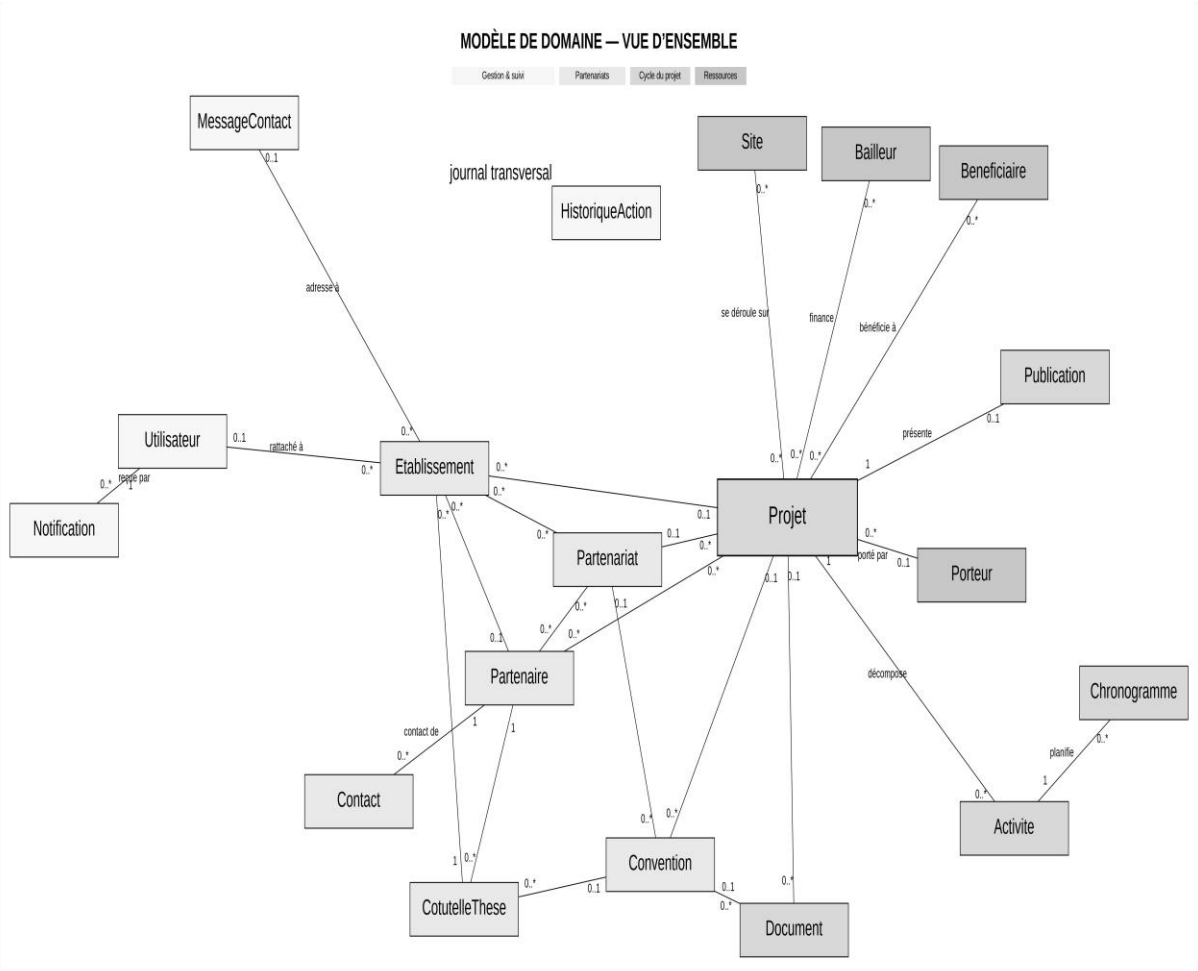


Figure 5.3 : Modèle de domaine (vue d'ensemble)

Chapitre 6 : Conception détaillée

Ce chapitre détaille l'architecture de l'application, les diagrammes de séquence, de classes, de paquetage et de déploiement. Il présente aussi les principes de conception de l'interface, qui constituent le cœur de la contribution de l'auteur.

6.1. Architecture de l'application

6.1.1. Architecture en trois niveaux

La plateforme suit une architecture client-serveur en trois niveaux, avec une séparation nette entre le frontend et le backend : le niveau présentation est une application monopage (SPA) en React 19, Vite et TypeScript qui s'exécute dans le navigateur ; le niveau métier est une API REST en NestJS 11, organisée en modules et documentée avec Swagger ; le niveau données est une base PostgreSQL 16, accessible via TypeORM, dont le schéma évolue par des migrations versionnées. La communication se fait en HTTP, au format JSON. Les fichiers téléversés (documents, logos, photos) sont stockés dans un dossier du serveur et servis par l'API.

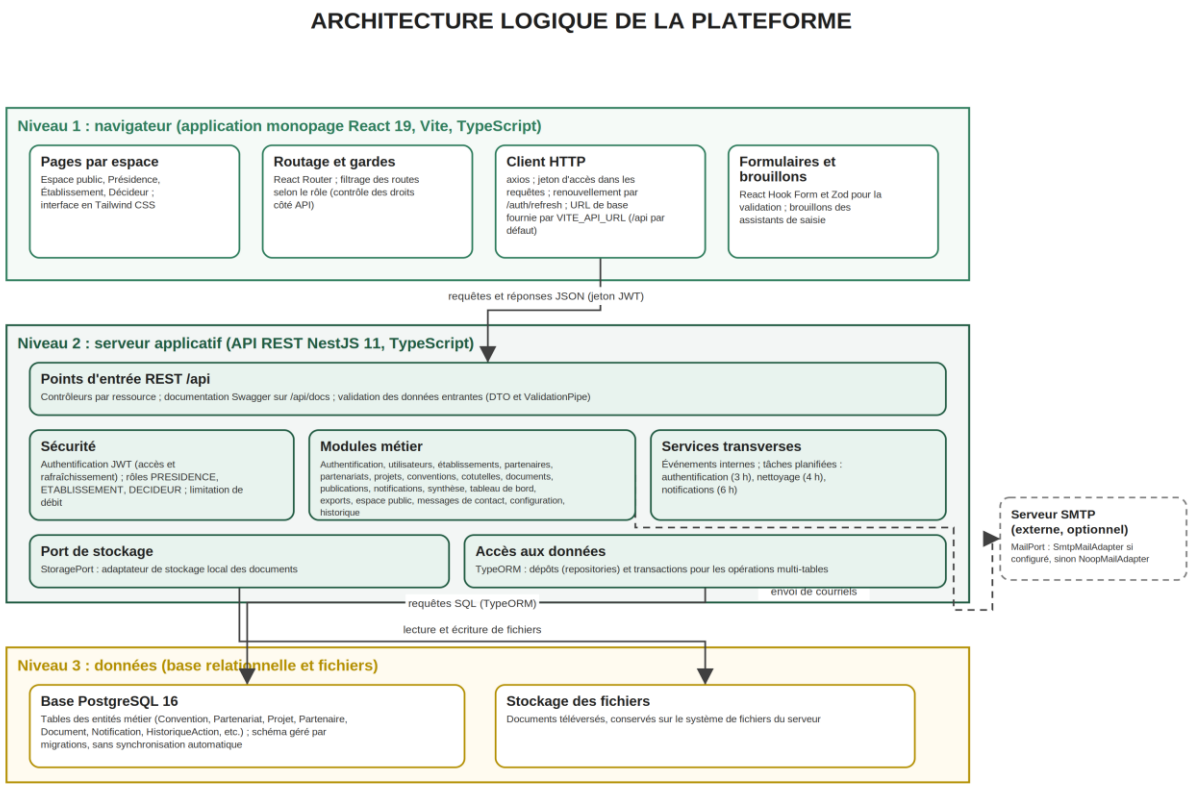


Figure 6.1 : Architecture logique de la plateforme

6.1.2. Mécanismes de communication

L'authentification repose sur un jeton JWT d'accès et un jeton de rafraîchissement.

Le client HTTP (Axios) ajoute automatiquement le jeton aux requêtes, transforme les chaînes vides en valeurs nulles, et tente un rafraîchissement du jeton en cas de réponse HTTP 401.

Un point d'accès de santé (GET /api/health) permet de vérifier l'état du serveur.

En développement, le serveur Vite relaie les requêtes /api et /uploads vers le backend, ce qui évite les problèmes d'origines croisées.

6.1.3. Principes de sécurité

- **6.1.3.1. Authentification et jetons**

Les mesures côté serveur, conçues et mises en œuvre par le binôme en charge du backend, comprennent l'authentification par jetons JWT et le hachage des mots de passe. Côté frontend, les routes sont filtrées selon le rôle, comme décrit au chapitre 7. Le contrôle effectif des droits repose toutefois sur l'API.

- **6.1.3.2. Protection des échanges**

Les requêtes sensibles sont soumises à une limitation du débit. Les en-têtes HTTP sont protégés et les origines autorisées sont configurées au moyen de CORS.

- **6.1.3.3. Réinitialisation du mot de passe**

La réinitialisation du mot de passe utilise un jeton de 32 octets encodé en base64url et stocké sous forme hachée. Le champ utilise_le garantit son usage unique. Le jeton est valable 30 minutes par défaut, selon la variable MOT_DE_PASSE_RESET_EXPIRATION_MINUTES. Les anciens jetons non utilisés sont invalidés.

Tableau 6.1 : Correspondance entre exigences et choix d'architecture

Exigence	Choix d'architecture
Rôle déterminant l'espace et les droits	Authentification par jeton ; routes protégées selon le rôle côté frontend ; contrôle des droits côté API
Séparation des données entre établissements	Rattachement automatique à l'établissement du compte connecté
Maintenabilité et évolutivité	Séparation en niveaux ; code typé (TypeScript) ; organisation modulaire
Déploiement reproductible	Conteneurs Docker (frontend, API, base)
Notifications périodiques	Tâches planifiées exécutées côté serveur

Source : analyse de l'auteur à partir du cahier des charges et du dépôt.

6.1.4. Principes de conception de l'interface

Le cahier des charges impose une identité institutionnelle : palette verte (#2F7D5D) et orange (#F2B233), typographie Inter, rayons de bordure de 6 à 12 px. Le frontend propose un thème clair et un thème sombre. La conception vise la simplicité d'utilisation pour les établissements, à travers des formulaires découpés en étapes pour les entités complexes, avec sauvegarde du brouillon, des états explicites pour chaque écran (chargement, erreur, liste vide), une interface responsive et des attributs d'accessibilité (ARIA) sur les composants interactifs. Une recherche de doublon est proposée avant la création d'un partenaire ; un test la couvre (frontend/tests/recherche-doublon-partenaire.test.ts).

6.2. Diagrammes de séquence de conception

6.2.1. Authentification et rafraîchissement du jeton

Le diagramme met en jeu l'utilisateur, le frontend (React et Axios), l'API NestJS et la base de données. Il présente la connexion, le stockage des jetons et leur rafraîchissement après une réponse HTTP 401.

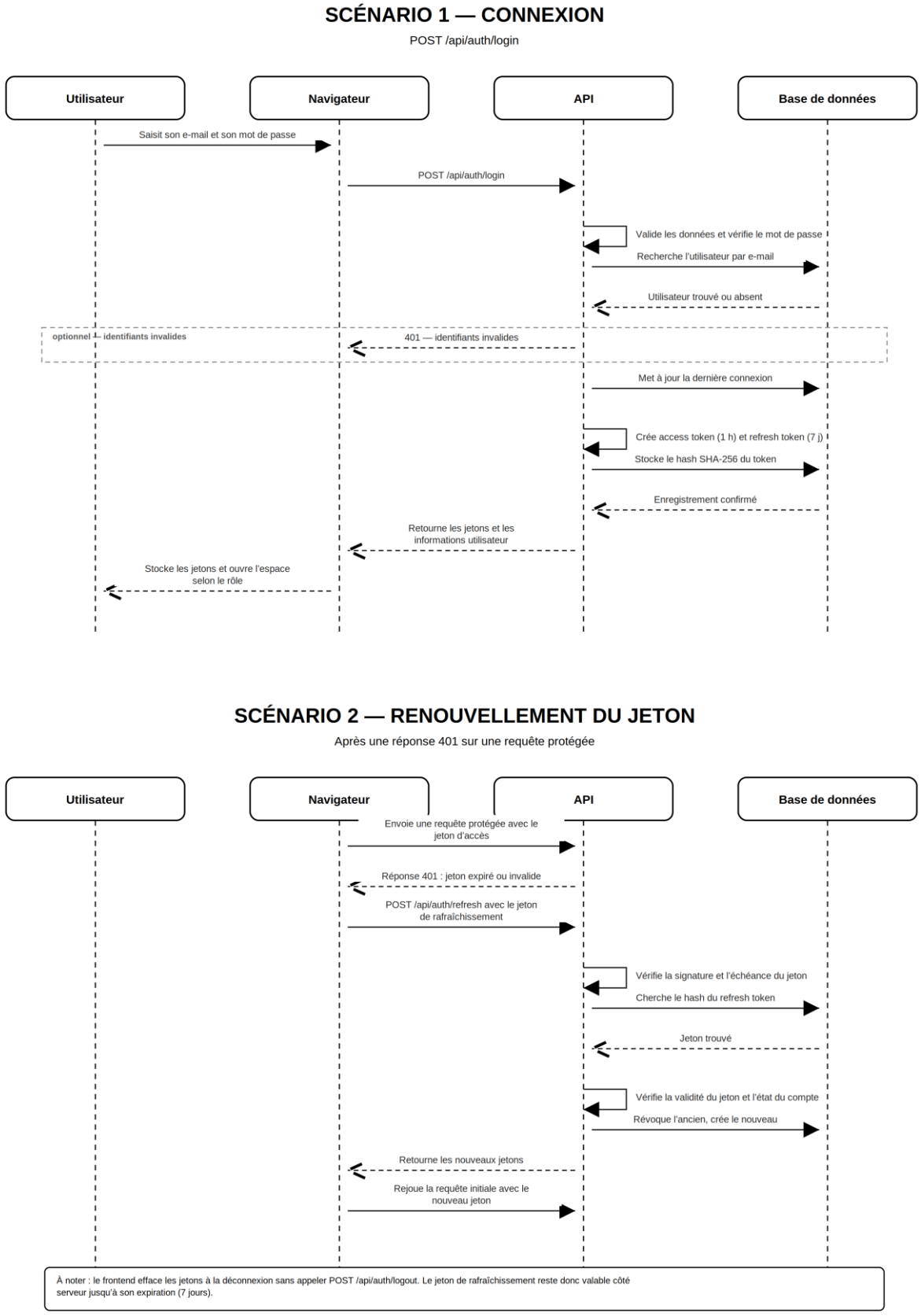


Figure 6.2 : Diagramme de séquence de l'authentification

- **6.2.1.1. Connexion**

1. L'utilisateur saisit ses identifiants dans le formulaire de connexion.

2. Le frontend envoie une requête de connexion à l'API.

3. L'API vérifie les identifiants dans la base, puis retourne un jeton d'accès et un jeton de rafraîchissement.

- **6.2.1.2. Stockage des jetons**

4. Le frontend conserve le jeton d'accès en mémoire et le jeton de rafraîchissement dans le stockage du navigateur (stockage de session par défaut, stockage local si l'option « Se souvenir de moi » est activée), puis redirige l'utilisateur vers son espace selon son rôle.

- **6.2.1.3. Rafraîchissement du jeton**

5. À chaque requête, le client ajoute le jeton d'accès. Si l'API répond 401, le client demande de nouveaux jetons, puis rejoue la requête.

6.2.2. Circuit de traitement d'une convention

Le circuit de traitement interne d'une convention est suivi manuellement : Projet et Partenariat, Secrétaire principale, Direction des Affaires Juridiques, puis signature.

La figure 6.3 détaille, au niveau de la conception, la séquence de création d'une convention déjà présentée au niveau système à la section 5.6.



Figure 6.3 : Diagramme de séquence de la création d'une convention

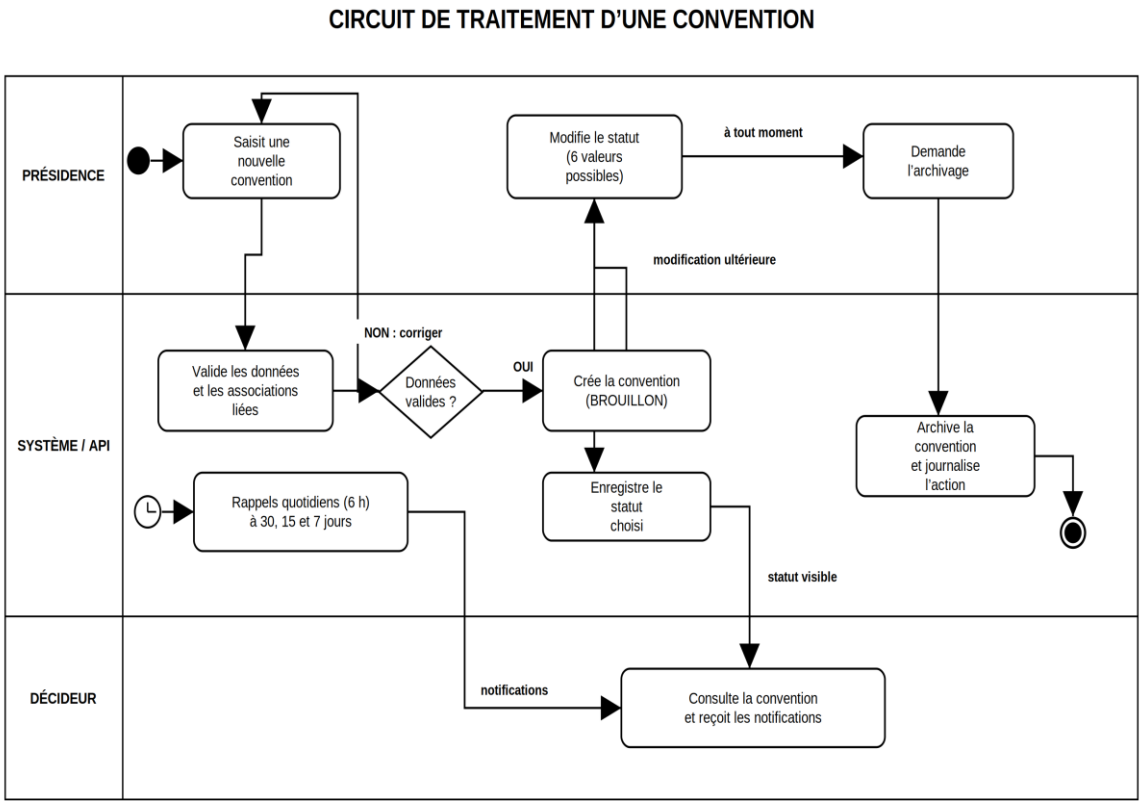


Figure 6.4 : Diagramme d'activités du circuit de traitement d'une convention

6.2.3. Diagrammes d'états-transitions

Une publication possède deux états, BROUILLON et PUBLIEE ; seule une publication PUBLIEE est visible des visiteurs. Une convention passe par les états BROUILLON, EN_ATTENTE_SIGNATURE, SIGNEE, ACTIVE, SUSPENDUE et EXPIREE, et la plateforme surveille ses dates de validité (seuils de 30, 15 et 7 jours). Un projet possède les états BROUILLON, EN_ATTENTE, EN_COURS, SUSPENDU, EXPIRE et TERMINE.

La figure 6.5 présente les états d'une publication (StatutPublication) ; seule une publication à l'état PUBLIEE est visible des visiteurs. Le trait pointillé désigne une transition à contrôler dans le code.

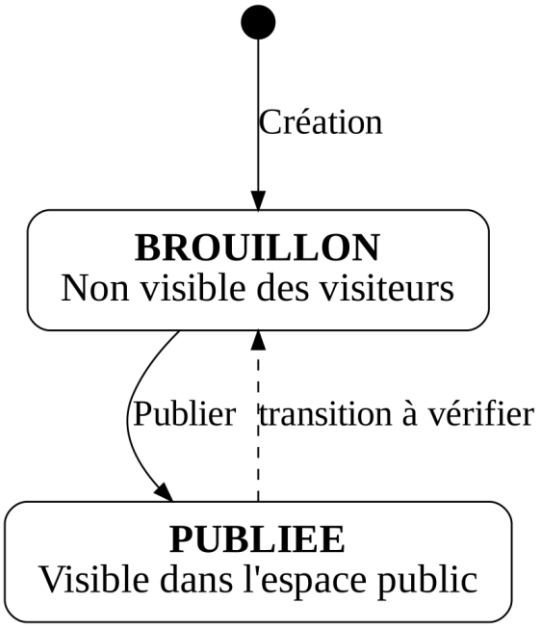


Figure 6.5 : Diagramme d'états d'une publication

La figure 6.6 présente les états d'une convention (StatutConvention), dans l'ordre de l'énumération du code. Les traits pointillés désignent des transitions à contrôler.

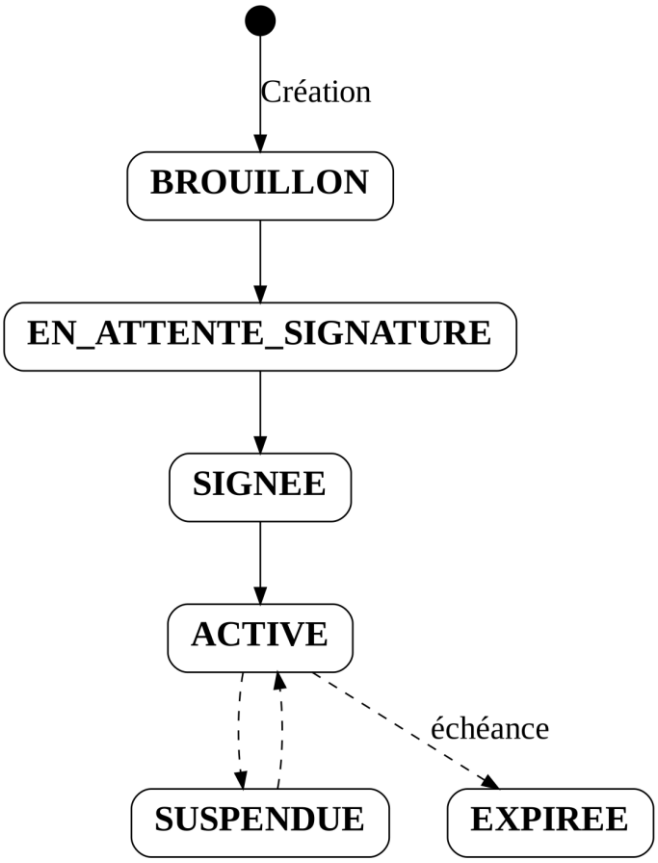


Figure 6.6 : Diagramme d'états d'une convention

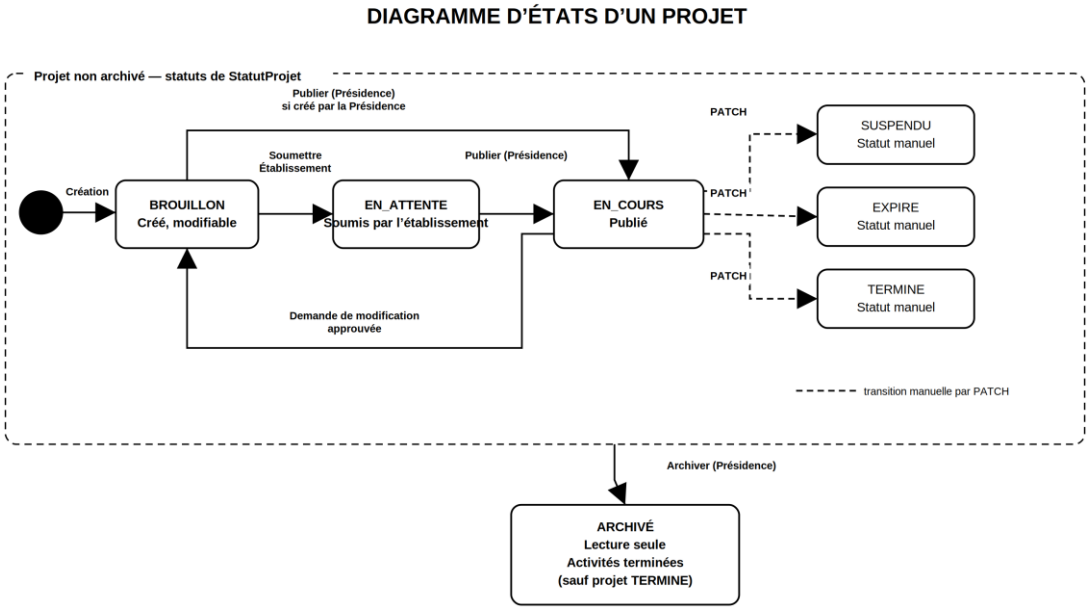


Figure 6.7 : Diagramme d'états d'un projet

6.3. Diagramme de classes

Le schéma relationnel est dérivé du diagramme de classes : chaque classe devient une table, les relations plusieurs-à-plusieurs passent par des tables d'association, et les rattachements facultatifs sont traduits par des clés étrangères acceptant la valeur nulle.

La figure 6.8 présente les classes du domaine « Projets » et leurs relations, d'après le tableau 5.6 et l'énumération StatutProjet ; les classes en pointillés appartiennent à un autre domaine.

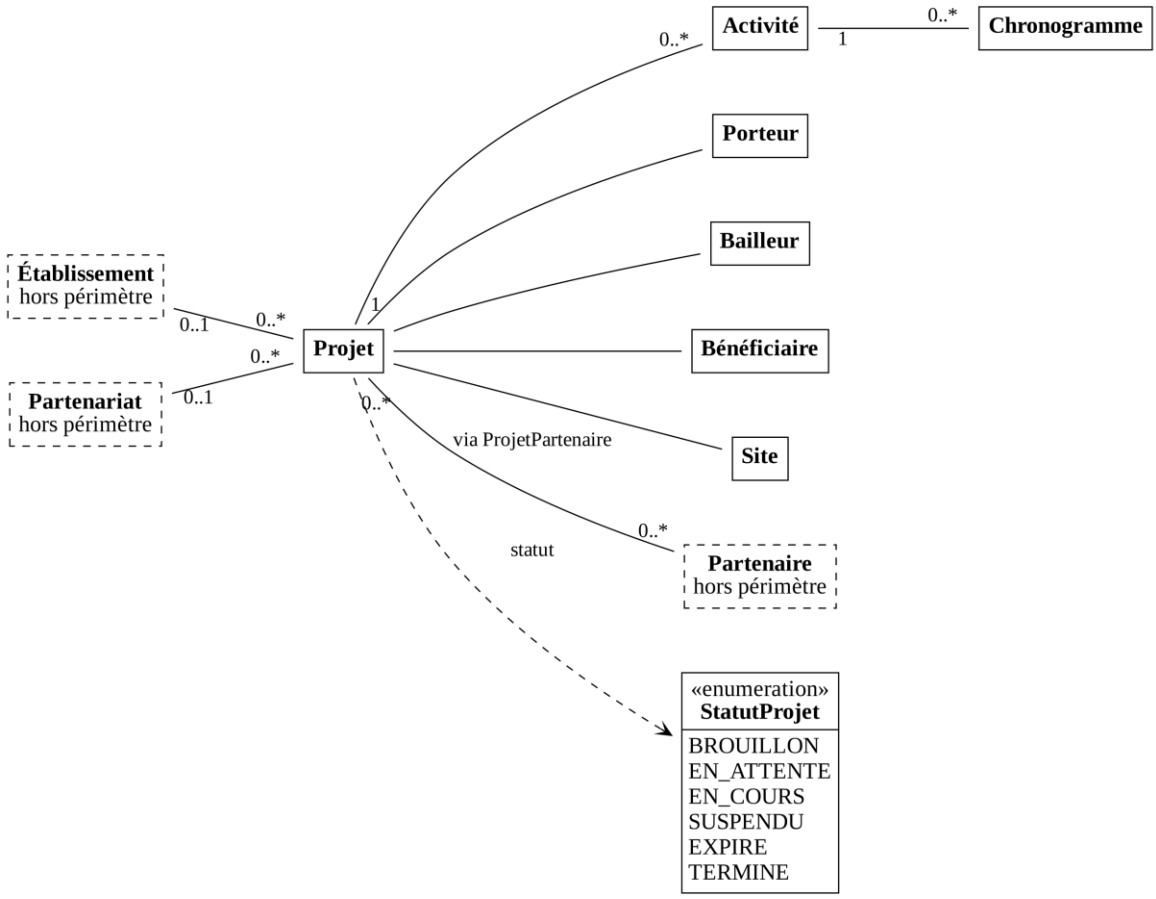


Figure 6.8 : Diagramme de classes du domaine « Projets »

La figure 6.9 présente les classes du domaine « Partenaires, partenariats et cotutelles », avec les énumérations StatutPartenaire et StatutPartenariat.

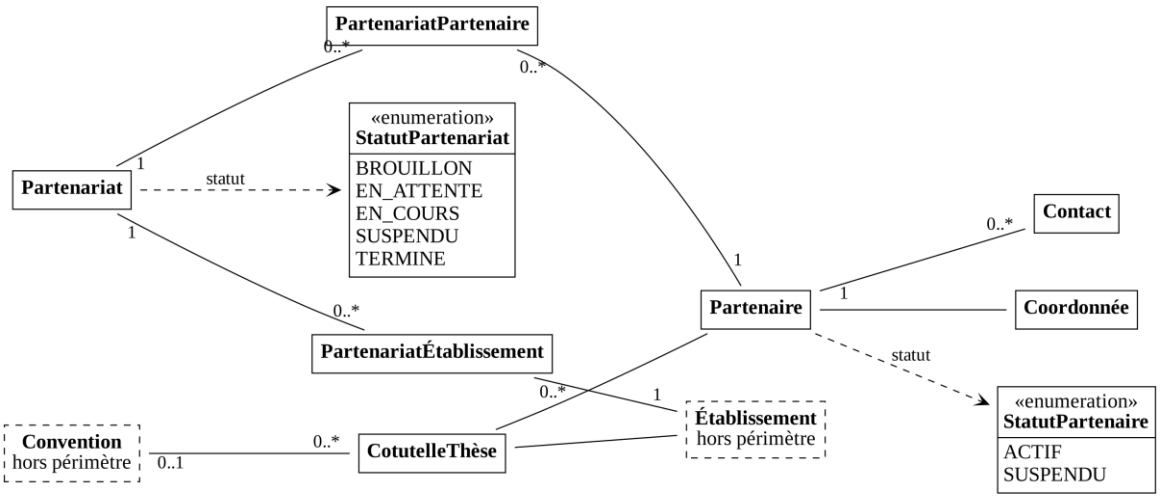


Figure 6.9 : Diagramme de classes du domaine « Partenaires, partenariats et cotutelles »

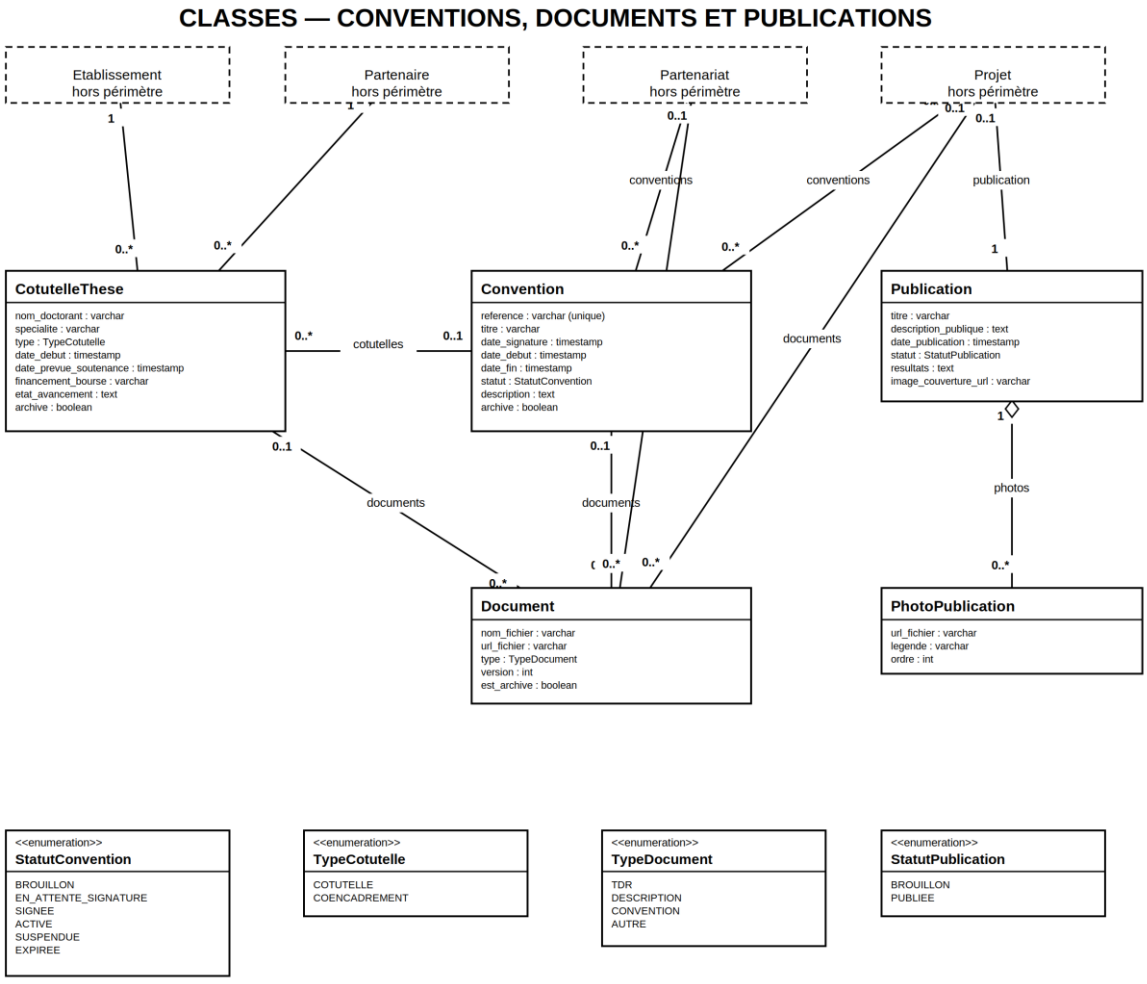


Figure 6.10 : Diagramme de classes du domaine « Conventions, documents et publications »

6.4. Diagramme de paquetage

Le frontend est organisé par fonctionnalités (features) : chaque domaine (projets, partenaires, conventions, publications, etc.) regroupe ses pages, composants, appels d'API et utilitaires. Les éléments communs sont isolés dans des dossiers partagés : components, hooks, lib et types. Ce découpage limite le couplage entre modules et facilite le travail en parallèle.

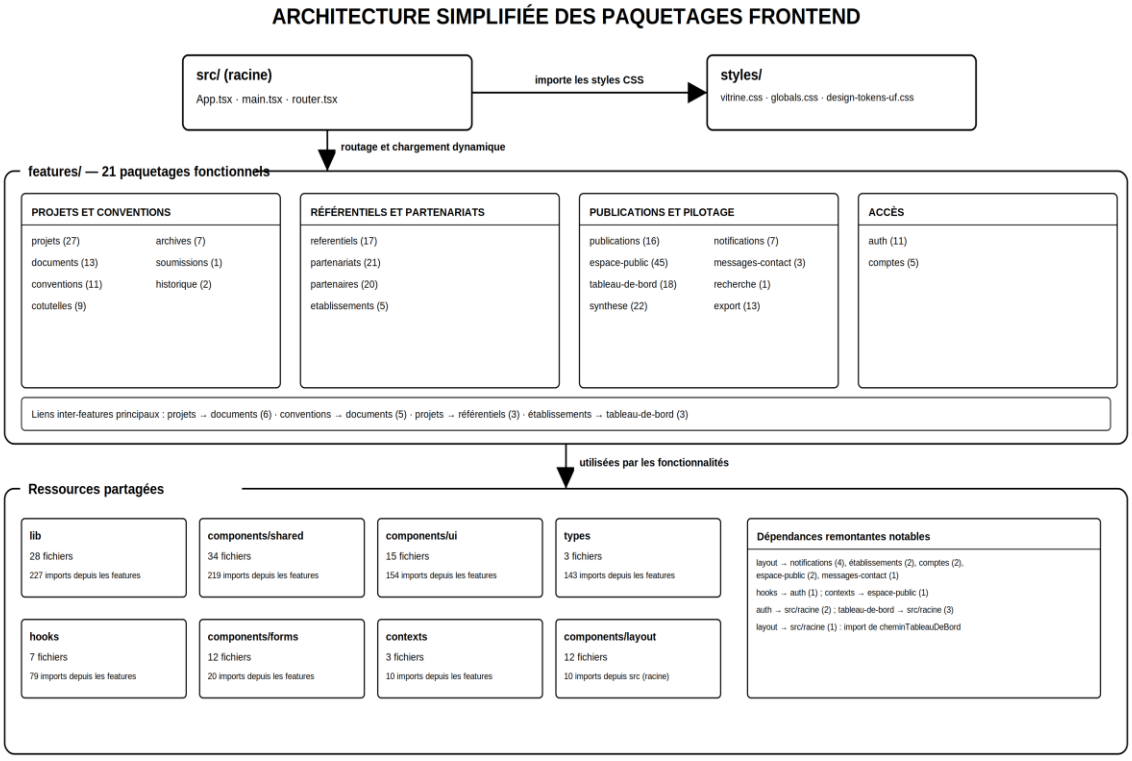


Figure 6.11 : Diagramme de paquetage du frontend

6.5. Diagramme de déploiement

Le déploiement logiciel repose sur Docker Compose. Trois services sont définis : la base PostgreSQL 16 (image postgres:16-alpine, données conservées dans un volume), le backend (image node:20-alpine, API sous le préfixe /api, documentation Swagger sous /api/docs, contrôle de santé GET /api/health) et le frontend (image node:22-alpine pour le développement et la compilation, puis nginx:1.27-alpine en production). Le démarrage est ordonné par depends_on avec la condition service_healthy : la base, puis le backend, puis le frontend. La figure 6.12 présente ce déploiement.

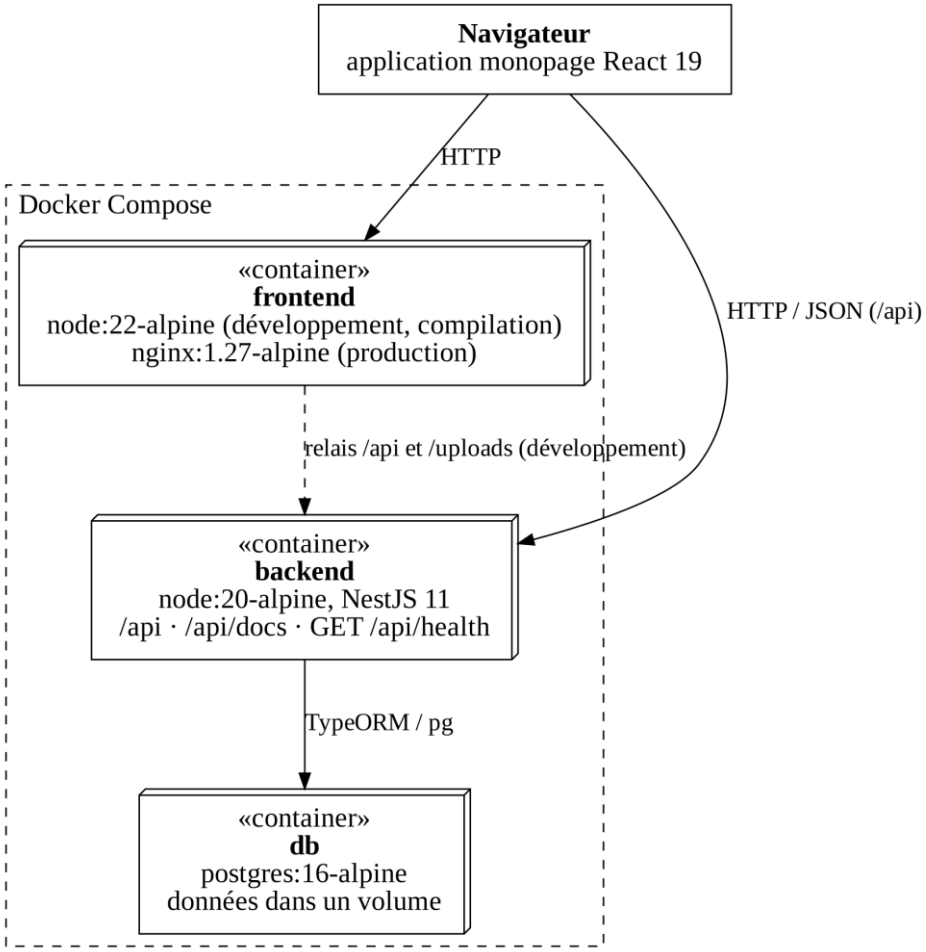


Figure 6.12 : Diagramme de déploiement logiciel (Docker Compose)

PARTIE 3 : MISE EN ŒUVRE ET RÉSULTATS

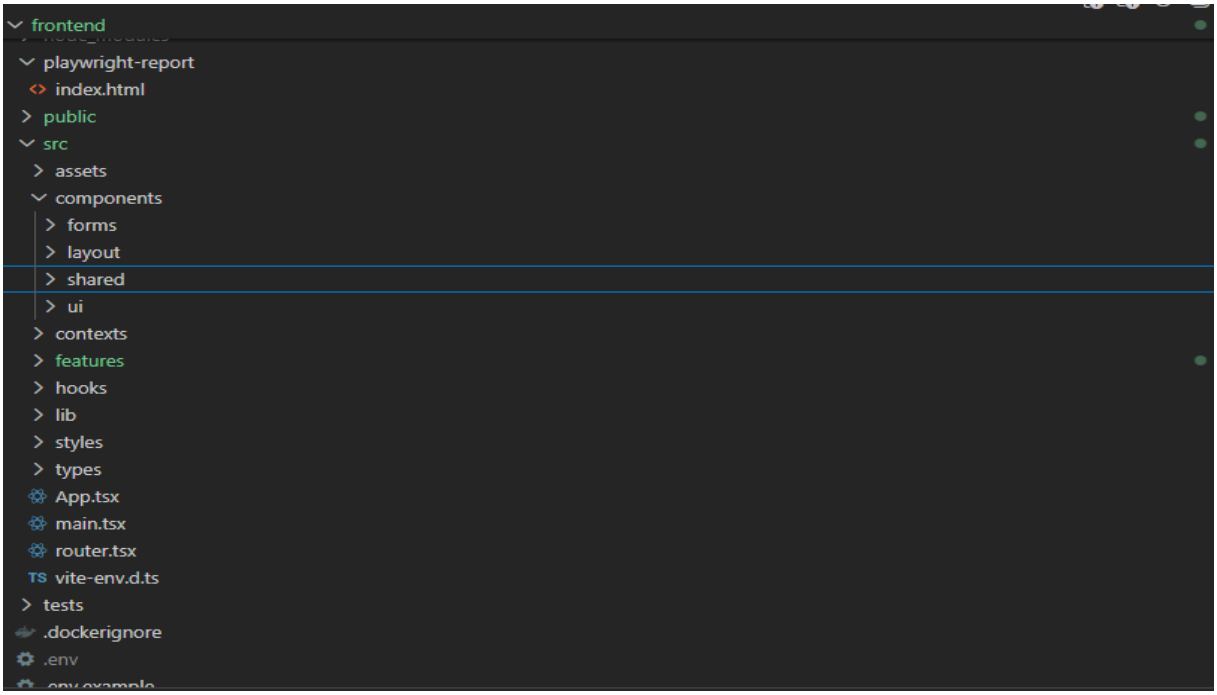
Chapitre 7 : Mise en œuvre

7.1. Architecture du système

Le présent mémoire traite l'architecture logicielle de la plateforme. L'architecture se compose de trois niveaux : présentation (frontend), logique métier (API) et données (PostgreSQL), décrits au chapitre 6.

7.1.1. Organisation des dossiers du frontend

Le code du frontend est organisé comme suit : features/ regroupe un dossier par domaine fonctionnel (archives, authentification, comptes, conventions, cotutelles, documents, établissements, notifications, partenaires, partenariats, projets, publications, synthèse, tableau de bord, etc.) ; components/ui contient les composants de base issus de shadcn/ui ; components/shared regroupe les composants métier réutilisables ; components/forms regroupe les formulaires et l'assistant par étapes ; components/layout regroupe les barres de navigation et le menu latéral ; hooks, lib et types regroupent les hooks, le client HTTP, les validateurs Zod et les types partagés.



7.1.2. Flux de données

Chaque fonctionnalité expose ses appels d'API dans un sous-dossier dédié. Les pages utilisent TanStack Query pour lire et modifier les données, ce qui centralise le cache et la gestion des

erreurs. Les formulaires reposent sur React Hook Form, validés par un schéma Zod ; les données sont ensuite transmises à l'API par le client HTTP commun.

7.2. Configuration des outils de conception

Les diagrammes ont été réalisés avec draw.io, les routes de l'API ont été testées avec Swagger, et le code est versionné avec Git. Swagger est exposé à l'adresse /api/docs, sous le préfixe /api de l'API.

7.3. Génération de la base de données

Le schéma est créé et modifié par des migrations TypeORM versionnées, présentées au chapitre 6. Elles sont exécutées avec « npm run migration:run », annulées avec « npm run migration:revert » et générées avec « npm run migration:generate ».

7.4. Configuration de l'environnement de développement

Le fichier docker-compose.yml lance les trois services : la base PostgreSQL 16 (image postgres:16-alpine), le backend et le frontend. Le frontend est construit à partir de l'image node:22-alpine, et l'image de production utilise nginx:1.27-alpine. Le backend utilise l'image node:20-alpine. Le serveur de développement relaie les requêtes /api et /uploads vers le backend. Le backend ne démarre qu'après la base, et le frontend après le backend.

7.5. Extraits de code des fonctionnalités principales

Les extraits ci-dessous sont simplifiés pour la lecture.

7.5.1. Intercepteur de requêtes

L'extrait 7.1 présente le principe de l'intercepteur du client HTTP : ajout du jeton d'accès, puis normalisation des champs vides en valeur nulle.

```ts
import axios, { type AxiosError, type InternalAxiosRequestConfig } from "axios"
import type { ReponseApi } from "@/types/api"
import { estPayloadNormalisable, normaliserPayload } from "./payload"
import { tokenStorage } from "./token-storage"
import { API_BASE_URL } from "./url"

let accessToken: string | null = null

export function definirAccessToken(token: string | null): void {
  accessToken = token
}       You, last month • feat: ajout de tsconfig dans dockerfile

export function obtenirAccessToken(): string | null {
  return accessToken
}

…
interface RequeteAvecRetry extends InternalAxiosRequestConfig {
  _retry?: boolean
}

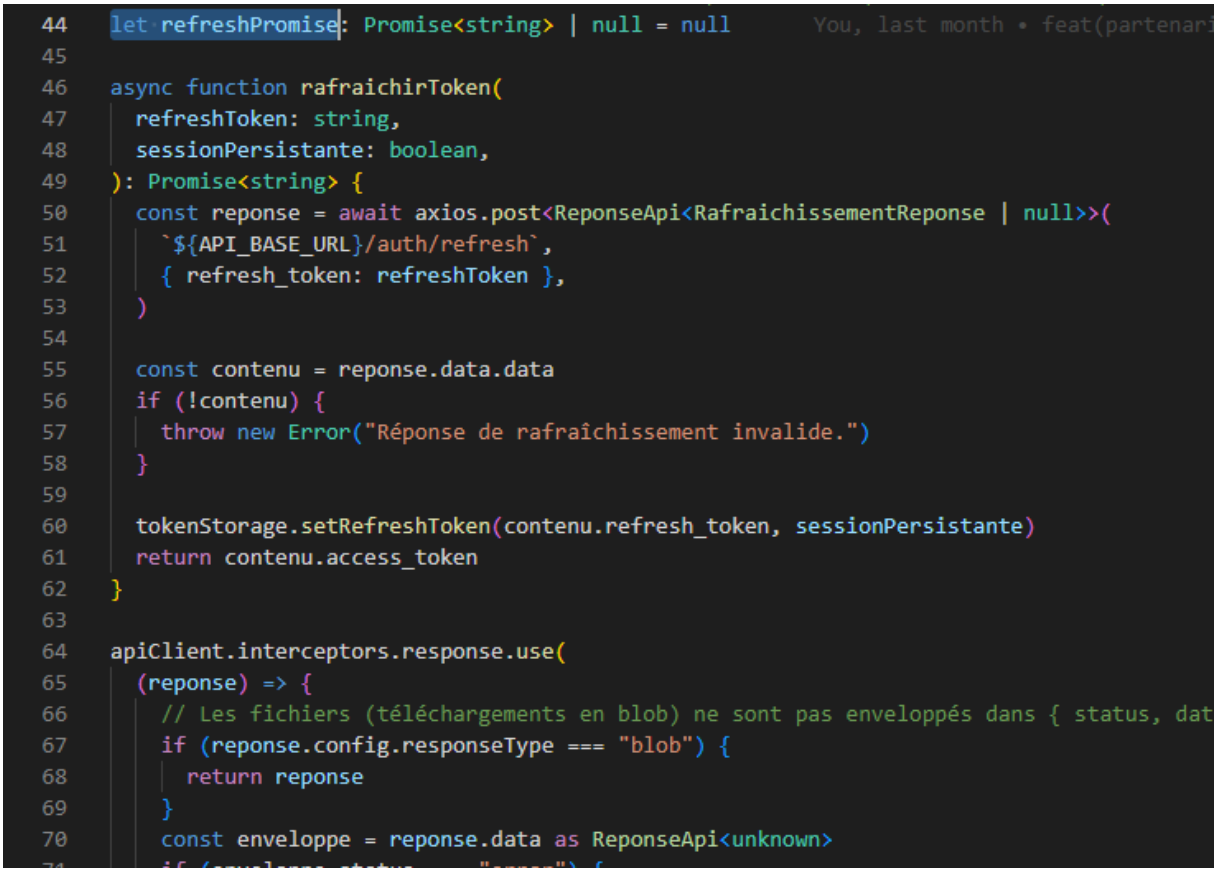
…
interface RafraichissementReponse {
  access_token: string
  refresh_token: string
}

const apiClient = axios.create({ baseURL: API_BASE_URL })
```

Extrait 7.1 : principe de l'intercepteur du client HTTP (fichier frontend/src/lib/api-client.ts). Extrait simplifié.

7.5.2. Rafraîchissement unique du jeton

Lorsque plusieurs requêtes échouent en même temps, un seul rafraîchissement de jeton est lancé, grâce à une promesse partagée. Cela évite d'épuiser le jeton de rafraîchissement.



```
let refreshPromise: Promise<string> | null = null      You, last month • feat(partenari

async function rafraichirToken(
  refreshToken: string,
  sessionPersistante: boolean,
): Promise<string> {
  const reponse = await axios.post<ReponseApi<RafraichissementReponse | null>>(
    `${API_BASE_URL}/auth/refresh`,
    { refresh_token: refreshToken },
  )

  const contenu = reponse.data.data
  if (!contenu) {
    throw new Error("Réponse de rafraîchissement invalide.")
  }

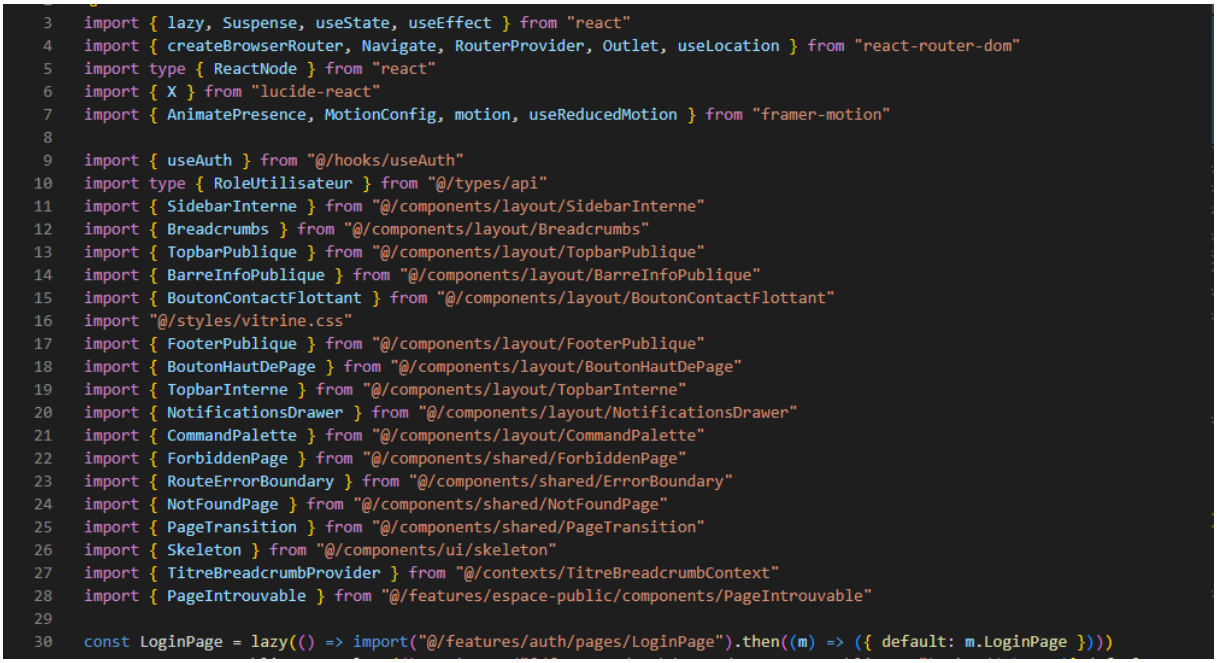
  tokenStorage.setRefreshToken(contenu.refresh_token, sessionPersistante)
  return contenu.access_token
}

apiClient.interceptors.response.use(
  (reponse) => {
    // Les fichiers (téléchargements en blob) ne sont pas enveloppés dans { status, dat
    if (reponse.config.responseType === "blob") {
      return reponse
    }
    const enveloppe = reponse.data as ReponseApi<unknown>
```

Extrait 7.2 : rafraîchissement unique du jeton (fichier frontend/src/lib/api-client.ts). Extrait simplifié.

7.5.3. Garde de routes par rôle

Le routage est assuré par React Router. L'application déclare plus de cent routes, publiques, d'authentification et propres à chaque rôle. Le composant de garde lit l'utilisateur connecté, redirige vers la connexion s'il n'y en a pas, affiche une page d'accès refusé si son rôle n'est pas autorisé, et affiche la page demandée sinon.



```
import { lazy, Suspense, useState, useEffect } from "react"
import { createBrowserRouter, Navigate, RouterProvider, Outlet, useLocation } from "react-router-dom"
import type { ReactNode } from "react"
import { X } from "lucide-react"
import { AnimatePresence, MotionConfig, motion, useReducedMotion } from "framer-motion"

import { useAuth } from "@/hooks/useAuth"
import type { RoleUtilisateur } from "@/types/api"
import { SidebarInterne } from "@/components/layout/SidebarInterne"
import { Breadcrumbs } from "@/components/layout/Breadcrumbs"
import { TopbarPublique } from "@/components/layout/TopbarPublique"
import { BarreInfoPublique } from "@/components/layout/BarreInfoPublique"
import { BoutonContactFlottant } from "@/components/layout/BoutonContactFlottant"
import "@/styles/vitrine.css"
import { FooterPublique } from "@/components/layout/FooterPublique"
import { BoutonHautDePage } from "@/components/layout/BoutonHautDePage"
import { TopbarInterne } from "@/components/layout/TopbarInterne"
import { NotificationsDrawer } from "@/components/layout/NotificationsDrawer"
import { CommandPalette } from "@/components/layout/CommandPalette"
import { ForbiddenPage } from "@/components/shared/ForbiddenPage"
import { RouteErrorBoundary } from "@/components/shared/ErrorBoundary"
import { NotFoundPage } from "@/components/shared/NotFoundPage"
import { PageTransition } from "@/components/shared/PageTransition"
import { Skeleton } from "@/components/ui/skeleton"
import { TitreBreadcrumbProvider } from "@/contexts/TitreBreadcrumbContext"
import { PageIntrouvable } from "@/features/espace-public/components/PageIntrouvable"

const LoginPage = lazy(() => import("@/features/auth/pages/LoginPage").then((m) => ({ default: m.LoginPage })))
```

Extrait 7.3 : garde de routes par rôle (fichier frontend/src/router.tsx). Extrait simplifié.

7.5.4. Validation d'un formulaire de convention

Chaque formulaire s'appuie sur un schéma Zod placé dans lib/validators. Les trois identifiants de rattachement (partenariat, projet, cotutelle) sont facultatifs et indépendants, conformément à la règle métier consolidée. La règle de dates impose une date de fin postérieure ou égale à la date de début.



```
import { z } from "zod"

import { jourDe, validerDates } from "./regles-dates"

const schemaConventionBase = z.object({
  titre: z.string().min(3, "Le titre doit contenir au moins 3 caractères"),
  date_signature: z.string().optional(),
  date_debut: z.string().optional(),
  date_fin: z.string().optional(),
  statut: z.enum(["BROUILLON", "EN_ATTENTE_SIGNATURE", "SIGNEE", "ACTIVE", "SUSPENDUE", "EXPIREE"]),
  description: z.string().optional(),
  partenariat_id: z.string().optional(),
  projet_id: z.string().optional(),
  cotutelle_id: z.string().optional(),
})

Yastellin, 3 weeks ago | author (Yastellin)
export interface OptionsSchemaConvention {
  // A l'ajout seulement : la date de debut ne peut pas etre anterieure au jour.
  exigerDateDebutFuture?: boolean
}

export function creerSchemaConvention({ exigerDateDebutFuture = false }: OptionsSchemaConvention = {}) {
  return schemaConventionBase.superRefine((valeurs, ctx) => {
    validerDates(ctx, {
      dateDebut: valeurs.date_debut,
      exigerDateDebutFuture,
```

Extrait 7.4 : schéma de validation d'une convention (fichier frontend/src/lib/validators/convention.schema.ts).

7.5.5. Normalisation des charges utiles

La fonction de normalisation remplace les chaînes vides par la valeur nulle, récursivement, avant l'envoi à l'API.

Extrait 7.5 : fonction normaliserPayload (fichier frontend/src/lib/payload.ts). Extrait simplifié.

7.5.6. Assistant par étapes et brouillon

Les entités complexes sont saisies dans un assistant par étapes, avec barre de progression. Le brouillon est sauvegardé dans le stockage local du navigateur et restauré à la réouverture du formulaire. Un utilisateur qui quitte la page par erreur ne perd donc pas sa saisie. Le brouillon est sauvegardé automatiquement toutes les 30 secondes (hook useWizardDraft, constante INTERVALLE_SAUVEGARDE_MS = 30000).

7.6. Difficultés rencontrées

Évolution de la règle sur les conventions : l'exigence initiale d'au moins un lien a été supprimée en cours de projet. Il a fallu supprimer la règle de validation correspondante, aligner le formulaire et vérifier le payload transmis à l'API.

Bug de soumission d'un formulaire : un défaut lié à React Hook Form a été relevé. Les valeurs étaient visibles dans la surveillance des champs, mais la fonction de soumission recevait un objet vide.

Champs vides : les formulaires HTML renvoient des chaînes vides, que l'API refuse pour certains types, comme les dates. La normalisation systématique en valeur nulle dans le client HTTP règle ce problème de façon centralisée.

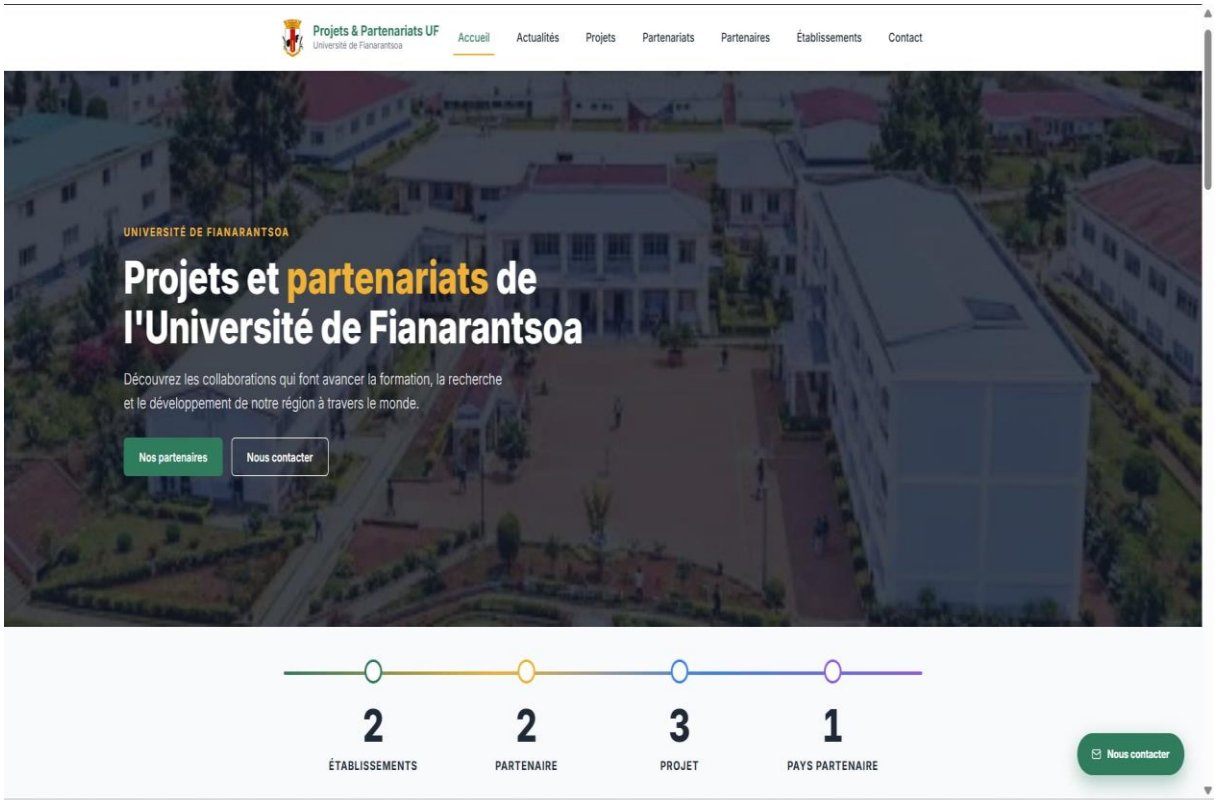
Coordination avec le backend : le contrat d'API a été ajusté tout au long du développement.

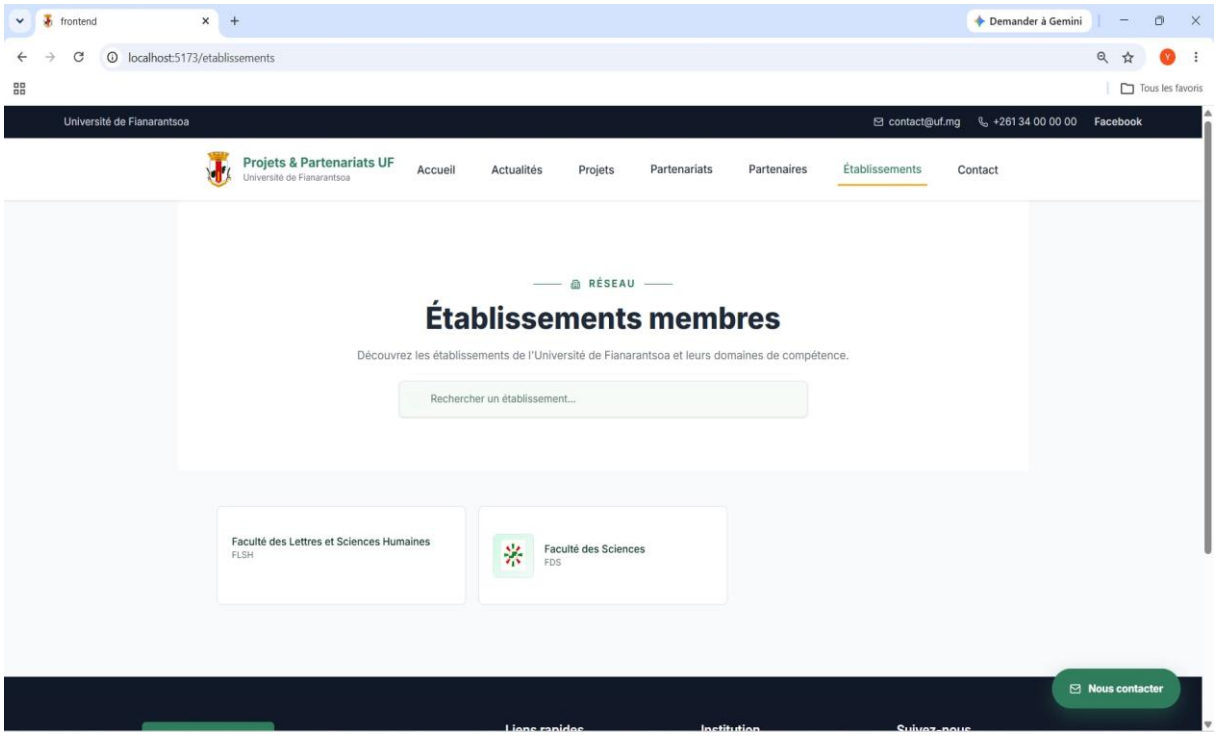
Chapitre 8 : Présentation de l'application

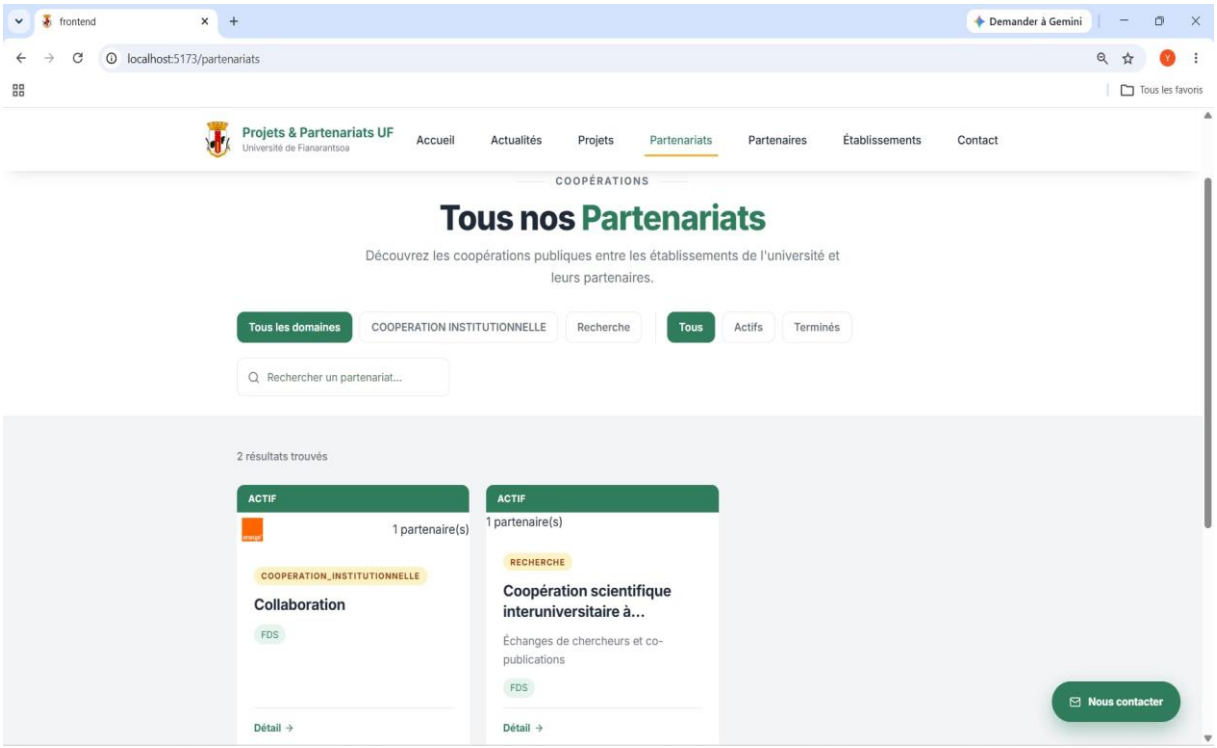
Ce chapitre présente l'application à travers des captures d'écran de ses espaces public et internes, ainsi que de son environnement de développement. Les images illustrent les principales pages et parcours disponibles.

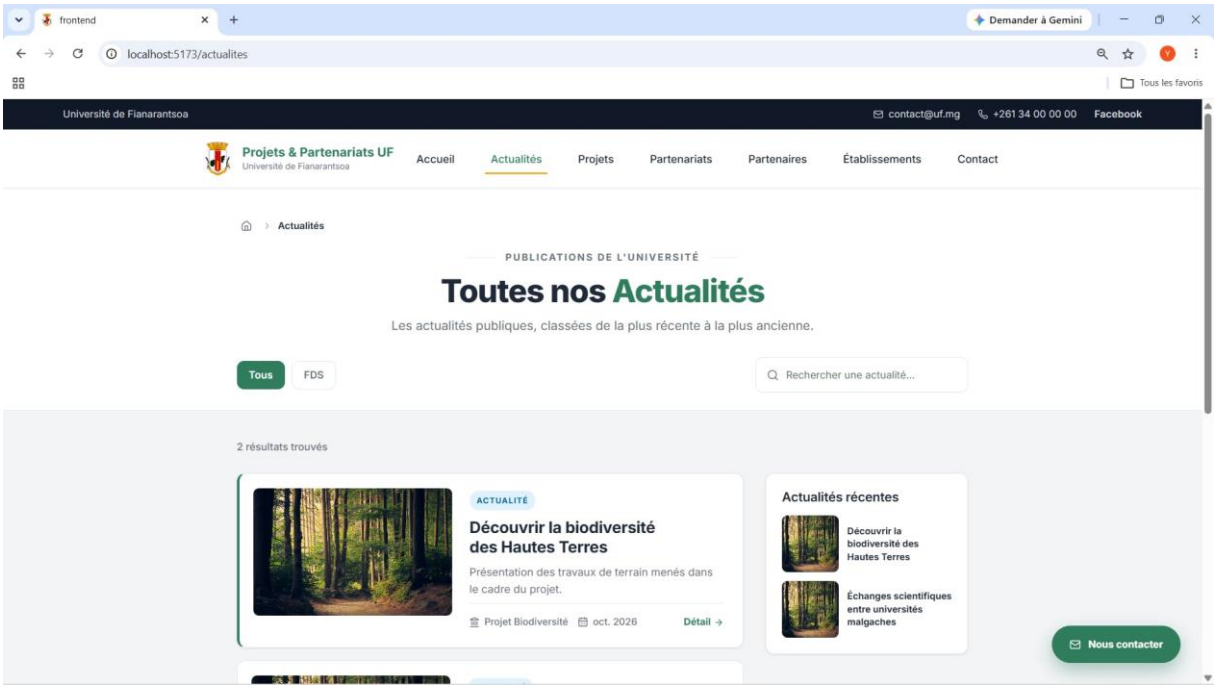
8.1. Espace public

L'espace public, accessible sans authentification, comporte les pages accueil, à propos, établissements, partenaires, projets, partenariats, actualités et contact. Il n'affiche que les publications au statut « Publiée ». La page d'accueil présente une bande de statistiques.


Projets & Partenariats UF
Université de Fianarantsoa
Accueil
Actualités
Projets
Partenariats
Partenaires
Établissements
Contact
UNIVERSITÉ DE FIANARANTSOA
Projets et partenariats de l'Université de Fianarantsoa
Découvrez les collaborations qui font avancer la formation, la recherche et le développement de notre région à travers le monde.
Nos partenaires
Nous contacter
2
ÉTABLISSEMENTS
2
PARTENAIRE
3
PROJET
1
PAYS PARTENAIRE
Nous contacter


frontend
Demander à Gemini
localhost:5173/etablissements
Tous les favoris
Université de Fianarantsoa
contact@uf.mg
+261 34 00 00 00
Facebook
Projets & Partenariats UF
Université de Fianarantsoa
Accueil
Actualités
Projets
Partenariats
Partenaires
Établissements
Contact
RÉSEAU
Établissements membres
Découvrez les établissements de l'Université de Fianarantsoa et leurs domaines de compétence.
Rechercher un établissement...
Faculté des Lettres et Sciences Humaines
FLSH
Faculté des Sciences
FDS
Nous contacter


frontend
Demander à Gemini
localhost:5173/partenariats
Tous les favoris
Projets & Partenariats UF
Université de Fianarantsoa
Accueil
Actualités
Projets
Partenariats
Partenaires
Établissements
Contact
COOPÉRATIONS
Tous nos Partenariats
Découvrez les coopérations publiques entre les établissements de l'université et leurs partenaires.
Tous les domaines
COOPERATION INSTITUTIONNELLE
Recherche
Tous
Actifs
Terminés
Rechercher un partenariat...
2 résultats trouvés
ACTIF
1 partenaire(s)
COOPERATION_INSTITUTIONNELLE
Collaboration
FDS
Détail →
ACTIF
1 partenaire(s)
RECHERCHE
Coopération scientifique interuniversitaire à...
Échanges de chercheurs et co-publications
FDS
Détail →
Nous contacter


frontend
Demander à Gemini
localhost:5173/actualites
Tous les favoris
Université de Fianarantsoa
contact@uf.mg
+261 34 00 00 00
Facebook
Projets & Partenariats UF
Université de Fianarantsoa
Accueil
Actualités
Projets
Partenariats
Partenaires
Établissements
Contact
Actualités
PUBLICATIONS DE L'UNIVERSITÉ
Toutes nos Actualités
Les actualités publiques, classées de la plus récente à la plus ancienne.
Tous
FDS
Rechercher une actualité...
2 résultats trouvés
ACTUALITÉ
Découvrir la biodiversité des Hautes Terres
Présentation des travaux de terrain menés dans le cadre du projet.
Projet Biodiversité
oct. 2026
Détail →
Actualités récentes
Découvrir la biodiversité des Hautes Terres
Échanges scientifiques entre universités malgaches
Nous contacter

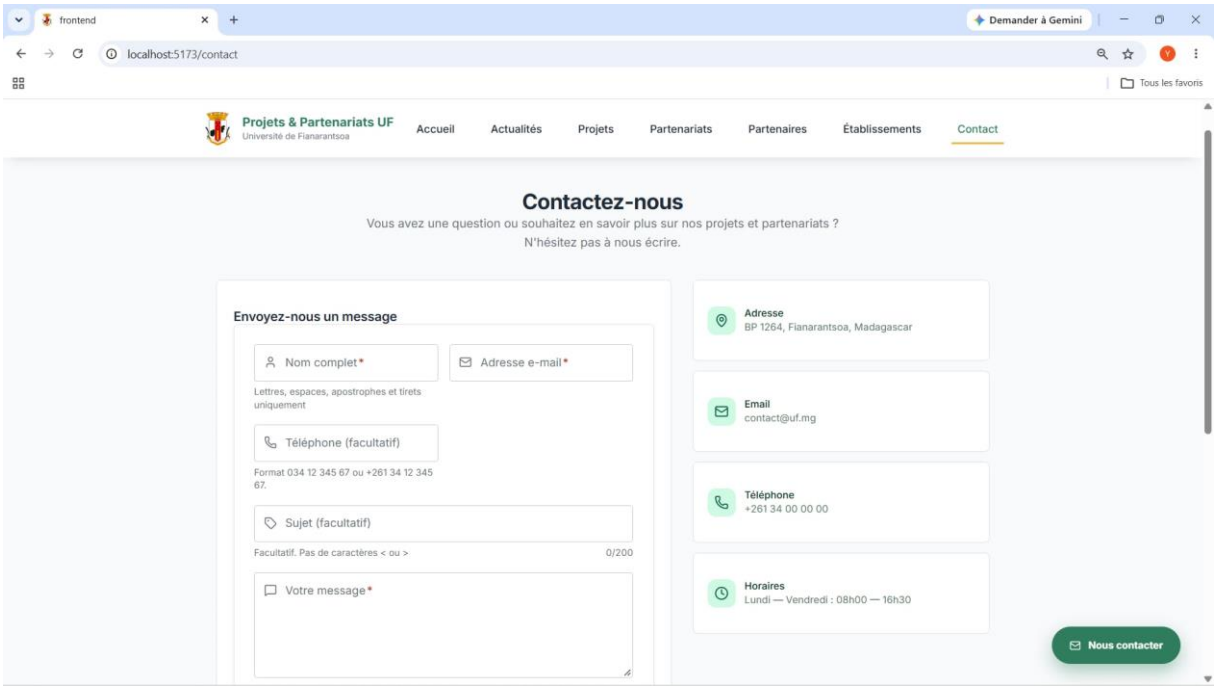


Figure 8.1 : Page d'accueil de l'espace public

8.2. Environnement matériel et déploiement logiciel

8.2.1. Poste de développement

Le développement a été réalisé sur un poste Windows. L'environnement de développement est exécuté par Docker Desktop, ce qui permet de lancer les trois services sans installer PostgreSQL ni Node.js sur le poste.

8.2.2. Conteneurs Docker

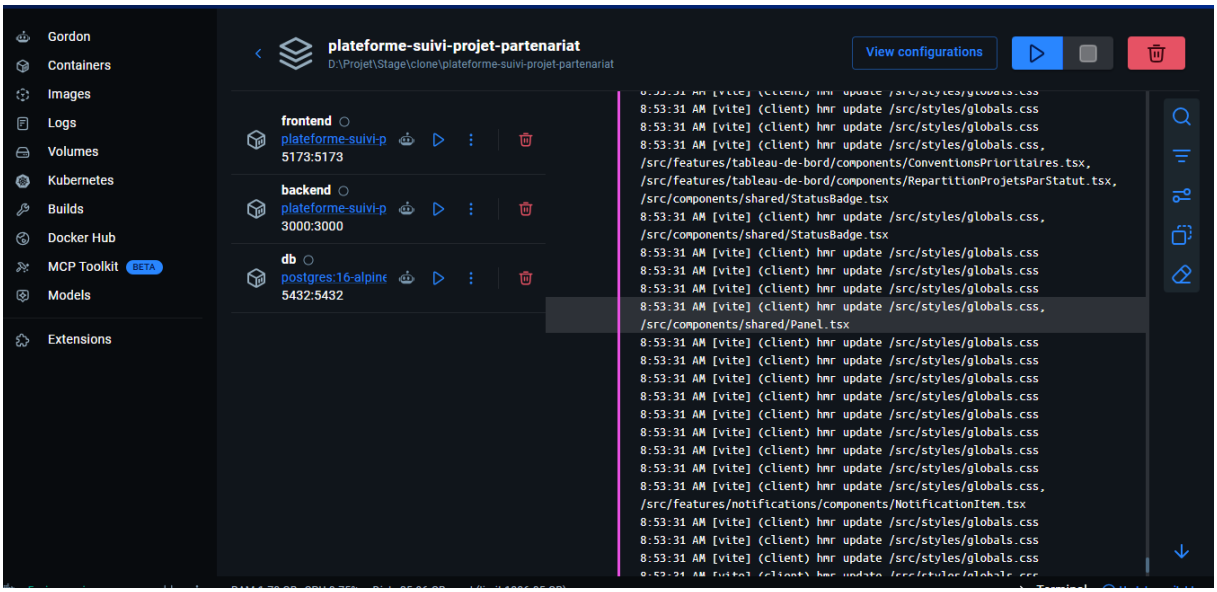


Figure 8.2 : Conteneurs Docker de l'environnement de développement

Tableau 8.1 : Conteneurs de l'environnement de développement

Service	Image ou technologie	Rôle	Dépendance au démarrage
frontend	Node 22 et Vite (développement) ; image de production Nginx 1.27	Application React servie au navigateur	Attend le backend en bonne santé
backend	Node 20 et NestJS	API REST sous le préfixe /api, documentation sous /api/docs	Attend la base en bonne santé
db	PostgreSQL 16 (Alpine)	Base de données relationnelle ; données conservées dans un volume	Aucune

Source : fichier docker-compose.yml et Dockerfiles du projet.

8.2.3. Ordre de démarrage

La base démarre en premier. Le backend attend que la base réponde à son contrôle de santé, puis le frontend attend que le backend soit prêt. Ce mécanisme est défini dans le fichier docker-compose.yml.

8.3. Authentification

La page de connexion permet d'accéder à l'espace de son rôle. Le jeton d'accès est conservé en mémoire, le jeton de rafraîchissement dans le stockage de session, ou dans le stockage local si l'option « Se souvenir de moi » est activée.

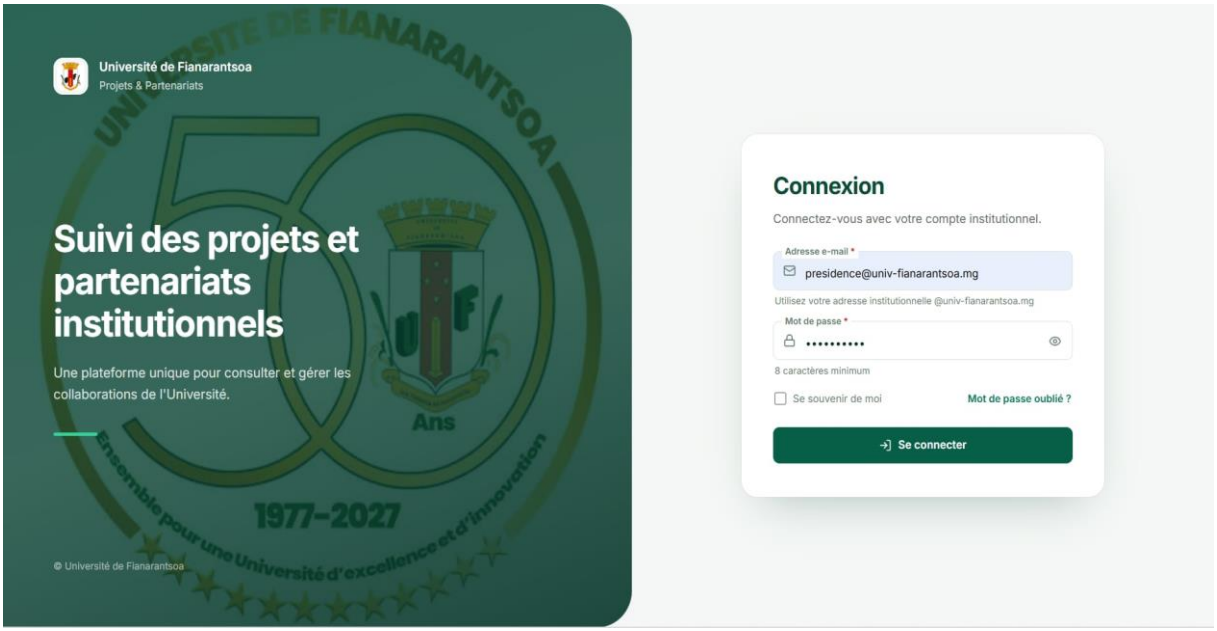


Figure 8.3 : Page de connexion

8.4. Espace Présidence

L'espace Présidence donne accès à la gestion complète : tableau de bord global, partenaires, partenariats, cotutelles, projets, conventions, documents, publications, archives, historique, synthèse annuelle, référentiels, établissements, exports et recherche.

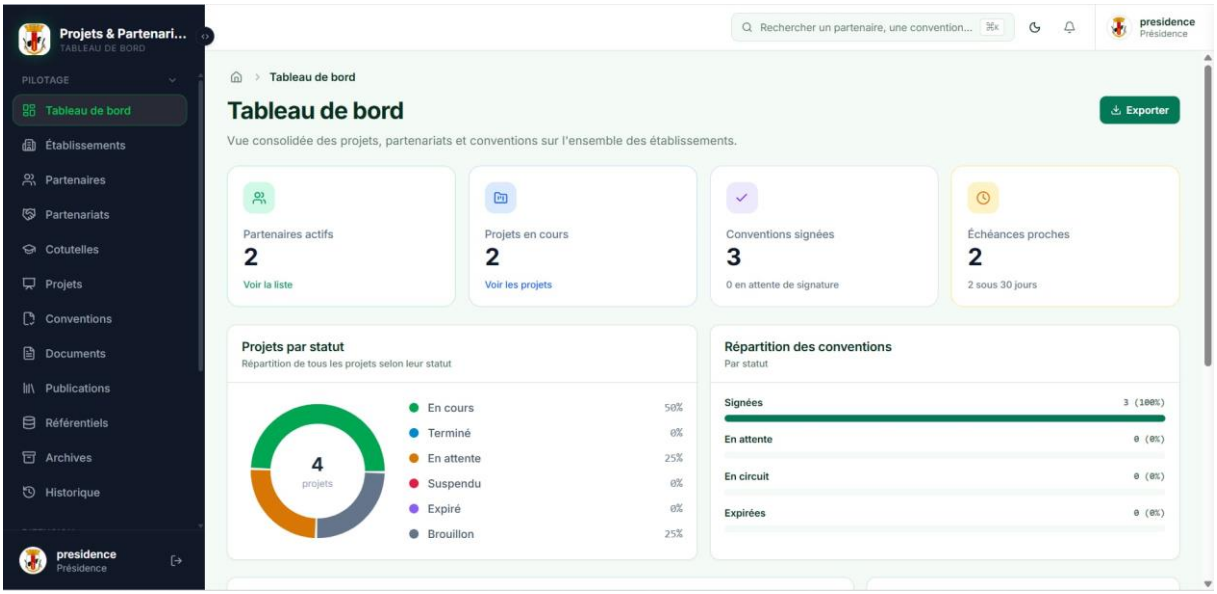


Figure 8.4 : Tableau de bord de la Présidence

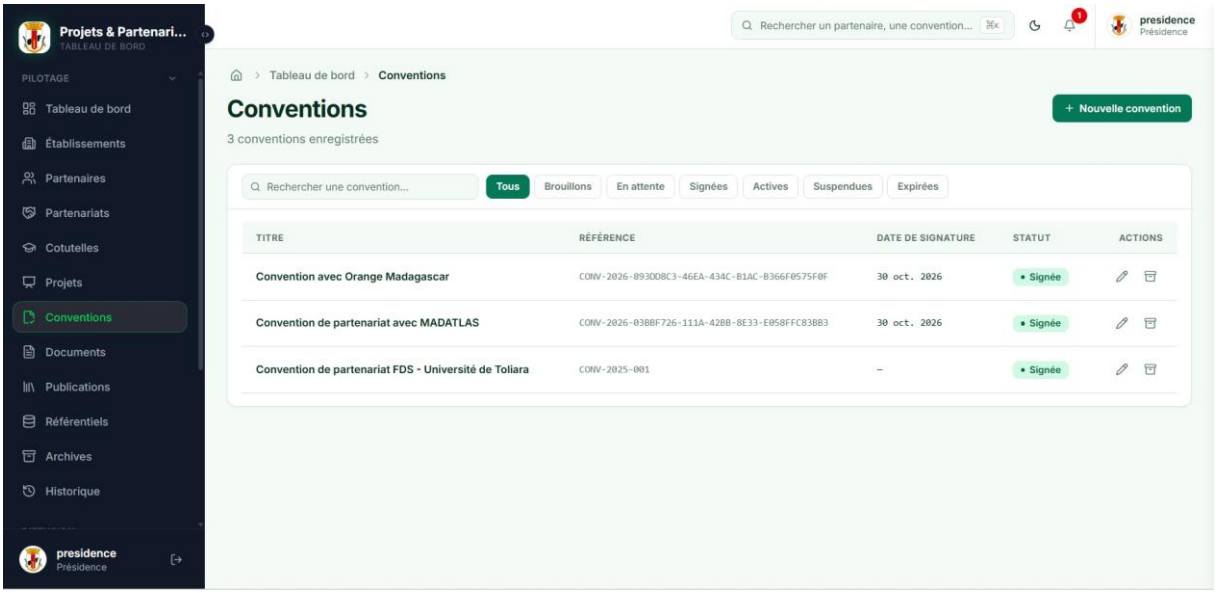


Figure 8.5 : Liste des conventions et formulaire de création

8.5. Espace Établissement

L'espace Établissement est limité aux données de l'établissement connecté : tableau de bord, partenaires, partenariats, projets, conventions, documents et publications. L'assistant de création d'un projet affiche une barre de progression par étapes.

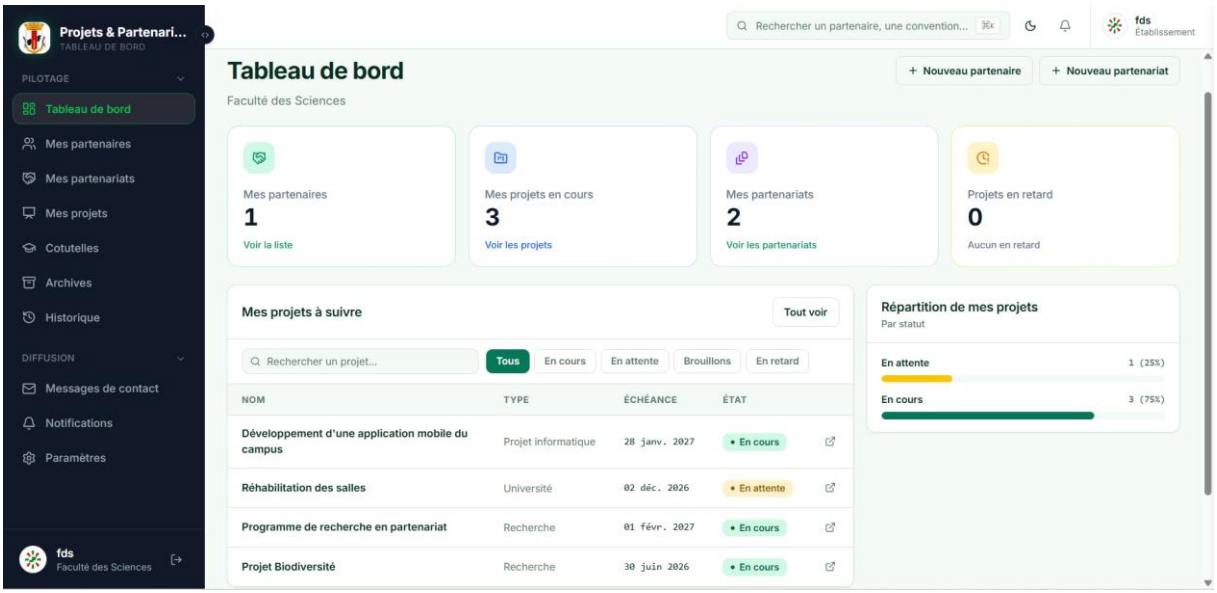


Figure 8.6 : Tableau de bord d'un établissement

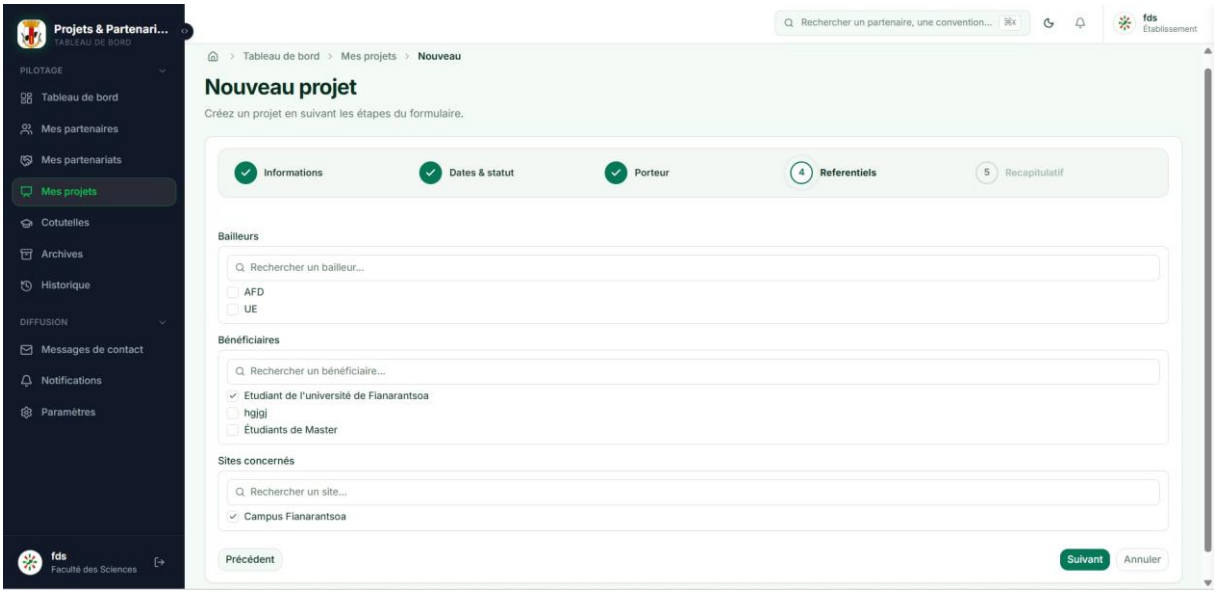


Figure 8.7 : Assistant de création d'un projet

8.6. Espace Décideur

L'espace Décideur propose un tableau de bord global, des indicateurs agrégés et l'accès en lecture seule aux fiches.

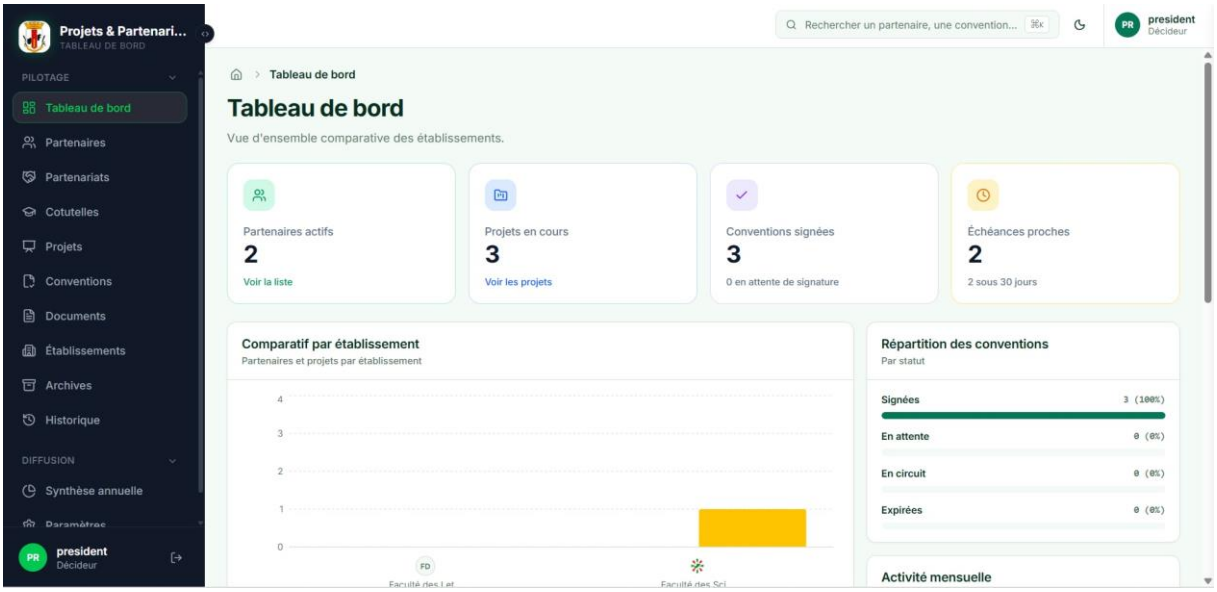


Figure 8.8 : Tableau de bord du Décideur

8.7. Synthèse annuelle, notifications et exports

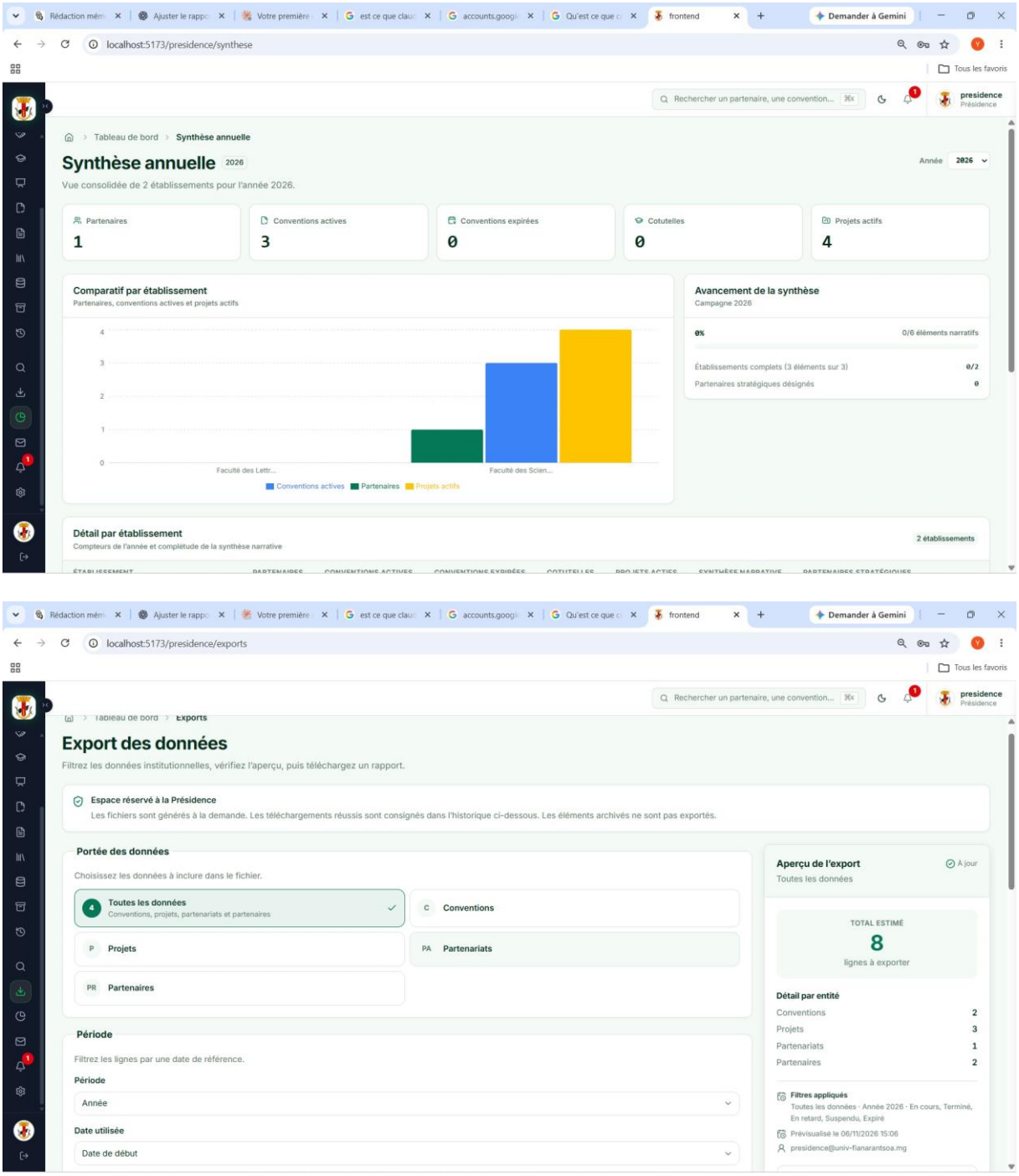


Figure 8.9 : Synthèse annuelle par établissement

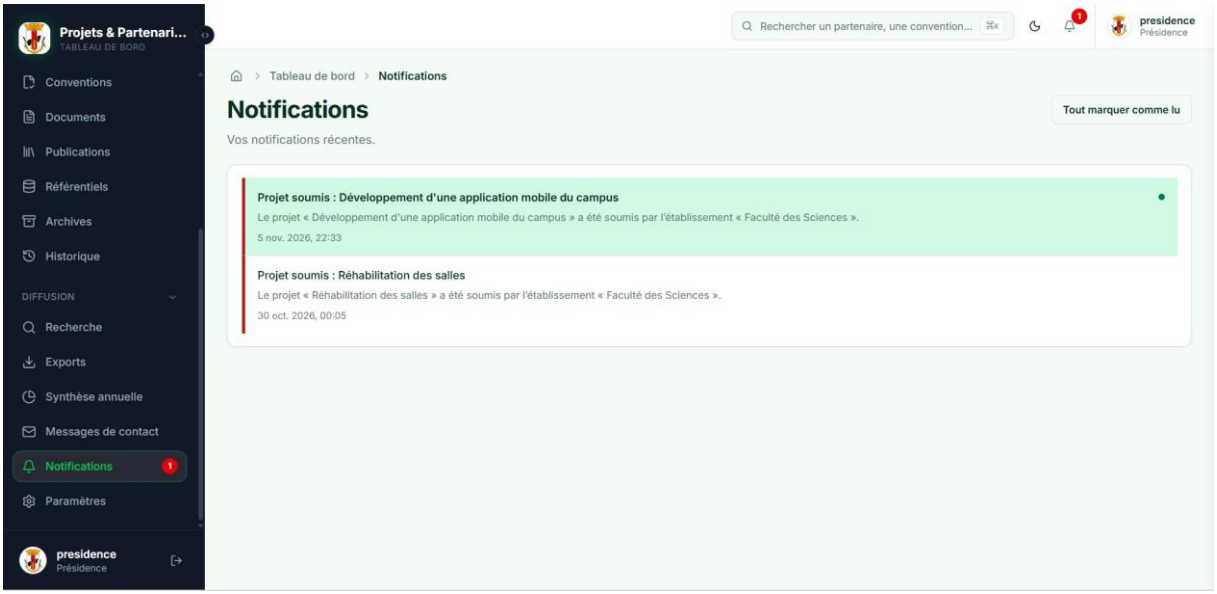


Figure 8.10 : Tiroir de notifications

8.8. Responsive design, thème et accessibilité

L'interface s'adapte aux écrans d'ordinateur et de téléphone. Un thème clair et un thème sombre sont proposés. L'accessibilité s'appuie sur les composants Radix UI, sur des attributs ARIA, et sur le respect de la préférence de mouvement réduit pour les animations.

Chapitre 9 : Évaluation et suggestions

9.1. Évaluation de l'application

9.1.1. Confrontation des objectifs et des résultats

Les résultats obtenus sont confrontés aux objectifs du chapitre 3 dans le tableau 9.1.

Tableau 9.1 : Confrontation des objectifs et des résultats

Objectif	Résultat constaté dans le dépôt	État
Gérer les partenaires, contacts et partenariats	Modules présents côté frontend et backend	Vérifié
Suivre les cotutelles	Module présent côté frontend et backend	Vérifié
Gérer les projets et les activités	Projets et activités présents	Vérifié
Suivre l'avancement en pourcentage	Aucun calcul trouvé dans le code ; archivage bloqué si des activités ne sont pas terminées (backend)	Non implémenté
Suivre le chronogramme	Modèle et migration dans le backend ; interface absente du frontend	Partiel (backend seul)
Gérer les conventions et documents	Modules présents ; rattachements facultatifs dans le schéma du frontend	Vérifié
Archiver et conserver l'historique	Archives et historique présents ; archivage par champ booléen	Vérifié
Synthèse annuelle	Module de synthèse avec graphique comparatif	Vérifié
Publications et espace public	Édition des publications et pages publiques présentes	Vérifié
Notifications	Notifications présentes ; tâche planifiée quotidienne côté serveur	Vérifié

Source : analyse du dépôt par l'auteur.

9.1.2. Tests automatisés

Le tableau 9.2 récapitule les fichiers de tests automatisés recensés dans le dépôt, en distinguant le frontend du backend, les outils utilisés et les emplacements concernés.

- **9.1.2.1. Tests du frontend**

Les tests de composants et unitaires du frontend utilisent Vitest et jsdom. Le dépôt en compte 51 fichiers : 50 dans frontend/tests et un dans frontend/src. Un fichier supplémentaire, sous frontend/tests/integration, couvre les tests d’intégration avec une configuration Vitest dédiée.

9.1.2.2. Tests du backend

Le backend comprend 18 fichiers de tests unitaires Jest sous backend/src (fichiers .spec.ts), ainsi que 16 fichiers de tests de bout en bout sous backend/test (fichiers .e2e-spec.ts), avec Jest, Supertest et PGlite. Ces tests portent sur l'API. Ils ne constituent pas des parcours automatisés dans le navigateur.

Tableau 9.2 : Tests automatisés présents dans le dépôt

Niveau	Outil	Périmètre	Nombre de fichiers
Tests de composants et unitaires (frontend)	Vitest, jsdom	frontend/tests et frontend/src	51 (50 dans frontend/tests, 1 dans frontend/src)
Tests d'intégration (frontend)	Vitest (configuration dédiée)	frontend/tests/integration	1
Tests unitaires (backend)	Jest	backend/src (fichiers .spec.ts)	18
Tests de bout en bout (backend)	Jest, Supertest, PGlite	backend/test (fichiers .e2e-spec.ts)	16

Source : dépôt du projet, recensement des fichiers de test par motif de nom. Les fichiers d'outils et de harnais ne sont pas comptés.

9.1.2.3. Limites des tests

Les nombres du tableau correspondent à un recensement de fichiers, et non à des résultats d'exécution. Les fichiers d'outils et de harnais sont exclus. Aucun test de bout en bout n'automatise un parcours dans le navigateur. Les résultats d'exécution sont présentés en annexe C.

9.1.3. Recette fonctionnelle

Les scénarios suivants sont proposés pour une recette fonctionnelle ultérieure. Ils n'ont pas été exécutés dans le cadre de cette évaluation. Ils couvrent notamment la création d'une convention avec zéro, un, deux ou trois rattachements, la modification d'un rattachement vers « Aucun », la vérification du payload transmis à l'API dans l'onglet réseau du navigateur et le contrôle des valeurs nulles en base.

9.1.4. Comparaison avec les solutions existantes

La comparaison porte sur les solutions envisagées au chapitre 4 (le tableau 4.2) : l'amélioration du suivi sous Excel, l'adoption d'un logiciel existant et la plateforme sur mesure. Cette comparaison est analytique : aucun logiciel existant n'a été évalué en pratique.

9.2. Contributions académiques et professionnelles

Sur le plan professionnel, le projet a abouti à une application web couvrant les principales fonctionnalités livrées : un frontend structuré par domaines, une gestion de session côté client, des formulaires validés par des schémas typés, des tableaux de bord et des composants réutilisables. L'auteur a travaillé sur le contrat d'API avec le binôme en charge du backend.

9.3. Limitations de l'étude et pistes de recherche futures

9.3.1. Limitations

L'interface du chronogramme des activités n'est pas disponible dans cette version, alors que son modèle de données existe.

Le calcul de l'avancement en pourcentage, exigé par le cahier des charges, n'est pas implémenté.

Il n'existe pas de tests de bout en bout dans le navigateur, et les tests d'accessibilité sont à approfondir.

La validation de l’interface interne auprès des utilisateurs finaux peut être approfondie à partir de scénarios de recette et de retours de terrain.

La reprise des données existantes sous Excel n'est pas traitée.

La cohérence entre convention et cotutelle reste à arbitrer (section 5.7).

9.3.2. Pistes futures

1. Compléter l'interface du chronogramme des activités, dont le modèle existe déjà.
2. Implémenter le calcul de l'avancement des projets en pourcentage, tel que défini par le cahier des charges.
3. Étendre les tests automatisés, notamment les tests de bout en bout dans le navigateur.
4. Approfondir l'accessibilité et la performance sur des connexions lentes.
5. Prévoir un import des données existantes sous Excel pour faciliter la transition.

CONCLUSION GÉNÉRALE

Bilan

Le présent mémoire a rendu compte de la conception et de la réalisation d'une plateforme de suivi des projets et partenariats de l'Université de Fianarantsoa, menée en binôme au cours d'un stage de trois mois. Face à un suivi réalisé sous Excel, la plateforme apporte une centralisation des informations, une séparation des données par établissement, une traçabilité par historique et archives, des notifications, une synthèse annuelle et un espace public.

La solution est une application web en trois niveaux : un frontend React et TypeScript, développé par l'auteur, une API REST NestJS et une base PostgreSQL, développées par le binôme. La démarche Agile a rythmé le développement, et UML a servi à modéliser le système. Les difficultés ont concerné l'évolution des règles de gestion, un défaut de soumission lié à un formulaire et la coordination avec le backend.

Perspectives

Les perspectives sont présentées à la section 9.3.2. Elles portent principalement sur le chronogramme, le calcul de l'avancement, les tests de bout en bout, l'accessibilité et la reprise des données existantes.

BIBLIOGRAPHIE

Booch G., Rumbaugh J. et Jacobson I. (1999). The Unified Modeling Language User Guide. Addison-Wesley.

Beck K. et al., 2001. *Manifeste pour le développement logiciel Agile* [en ligne]. Disponible sur : https://agilemanifesto.org/iso/fr/manifesto.html

Object Management Group. *Unified Modeling Language Specification* [en ligne]. Disponible sur : https://www.omg.org/spec/UML/

Université de Fianarantsoa, 2026. *Cahier des charges consolidé de la plateforme de suivi des projets et partenariats,* version 4 avec mise à jour convention (document interne, 31 octobre 2026).

WEBOGRAPHIE

Meta Platforms, *React : documentation officielle* [en ligne]. Disponible sur : https://react.dev

Microsoft, *TypeScript : documentation* [en ligne]. Disponible sur : https://www.typescriptlang.org/docs/

Vite. Documentation officielle. Disponible sur : https://vite.dev

Tailwind Labs. Tailwind CSS : documentation. Disponible sur : https://tailwindcss.com/docs

Radix UI et shadcn/ui. Documentation officielle. Disponible sur : https://www.radix-ui.com et https://ui.shadcn.com

React Router. Documentation officielle. Disponible sur : https://reactrouter.com

TanStack. TanStack Query : documentation. Disponible sur : https://tanstack.com/query

React Hook Form. Documentation officielle. Disponible sur : https://react-hook-form.com

Zod. Documentation officielle. Disponible sur : https://zod.dev

NestJS. Documentation officielle. Disponible sur : https://docs.nestjs.com

TypeORM. Documentation officielle. Disponible sur : https://typeorm.io

PostgreSQL Global Development Group. PostgreSQL 16 : documentation. Disponible sur : https://www.postgresql.org/docs/16/

Docker Inc. *Docker Compose : documentation* [en ligne]. Disponible sur : https://docs.docker.com/compose/

Vitest. *Documentation* [en ligne]. Disponible sur : https://vitest.dev

Jest. Documentation officielle. Disponible sur : https://jestjs.io

Axios. Documentation officielle. Disponible sur : https://axios-http.com

Recharts. *Documentation* [en ligne]. Disponible sur : https://recharts.org

diagrams.net (draw.io). *Documentation* [en ligne]. Disponible sur : https://www.drawio.com

ANNEXES

Les annexes sont paginées en continuité du rapport. Chaque annexe est annoncée dans le texte principal

Annexe A : Synthèse des règles de gestion

Cette annexe synthétise les principales règles de gestion du cahier des charges consolidé.

1. La Présidence dispose d'un compte unique pour l'équipe Projet et Partenariat.
2. Chaque établissement possède un compte unique rattaché à cet établissement.
3. Un ou plusieurs comptes Décideur peuvent exister sans rattachement à un établissement.
4. Un établissement ne gère que ses propres données.
5. L'équipe Projet et Partenariat dispose d'une vision globale.
6. Le nom et l'intitulé officiel d'un projet sont distincts.
7. Le budget, le type et le statut sont des attributs directs du projet.
8. Un projet peut avoir plusieurs activités, et chaque activité appartient à un seul projet.
9. L'avancement est calculé à partir des activités terminées et reste indépendant du chronogramme (exigence non implémentée).
10. Un projet peut avoir plusieurs sites et bénéficiaires.
11. Un partenariat peut concerner plusieurs établissements et partenaires.
12. Un partenariat peut regrouper plusieurs projets.
13. Un projet peut être rattaché à au plus un partenariat, ou à aucun.
14. Un projet peut avoir ses propres partenaires directs.
15. Une cotutelle est associée à un établissement et à un partenaire universitaire.
16. Une convention peut être liée à un partenariat, un projet, une cotutelle, à plusieurs de ces entités, ou à aucune.
17. Les champs de rattachement d'une convention sont optionnels et acceptent la valeur nulle.
18. L'archivage conserve les données et les anciennes versions.
19. Seules les publications publiées sont visibles publiquement.

20. Les informations publiques sont séparées des informations internes.

21. Le Décideur est en lecture seule.

22. Les notifications possèdent un état lu ou non lu.

23. Les contrôles d'échéance et de retard sont périodiques.

Annexe B : Extraits de code complémentaires

La vérification ci-dessous distingue les objets simples des tableaux et des valeurs nulles.

```
function estObjetSimple(valeur: unknown): valeur is Record<string, unknown> {
  return (
    typeof valeur === 'object' &&
    valeur !== null &&
    !Array.isArray(valeur)
  )
}
```

Extrait B.1 : fonction estObjetSimple (fichier frontend/src/lib/payload.ts). Extrait simplifié.

Annexe C : Résultats d'exécution des tests

Exécution du 6 novembre 2026, sandbox Linux, Node.js 22.22.3, npm 10.9.8 (commit a007303).

Frontend (npm run test, Vitest) : 51 fichiers de test, 246 tests réussis, 0 échec.

Backend, tests unitaires (npm run test, Jest) : 18 suites, 112 tests réussis, 0 échec.

Backend, tests de bout en bout (npm run test:e2e, Jest et PGlite) : 16 suites, 166 tests, dont 5 réussis et 161 en échec. Cause : la migration AddLogoUtilisateur (1794000000000) n'est pas listée dans le harnais de test (backend/test/harness/application-test.ts), d'où l'absence de la colonne logo_url. Les résultats e2e ne permettent donc pas de conclure sur le comportement de l'application à ce stade.

Table des matières

INTRODUCTION GÉNÉRALE 1
PARTIE 1 : PRÉSENTATION GÉNÉRALE 2
Chapitre 1 : Ecole de Management et d'Innovation Technologique 3
1.1. Présentation générale 3
1.2. Historique 3
1.3. Missions et formations 4
1.4. Organisation 9
Chapitre 2 : Présentation ITDC Mada 11
2.1. Historique 11
2.2. Services 11
2.3. Formations 12
2.4. Mission 12
2.5. Clients de l'ITDC MADA 12
2.6. Méthode de travail 13
2.7. Organigramme d'ITDC MADA 14
Chapitre 3 : Présentation du projet 15
3.1. Formulation du projet 15
3.2. Justification de l'importance du sujet 15
3.3. Objectifs et besoins utilisateurs 16
3.4. Moyens nécessaires à la réalisation du projet 18
3.5. Chronogramme de réalisation du projet 19
3.6. Résultat attendu 19
PARTIE 2 : ANALYSE ET CONCEPTION 20
Chapitre 4 : Analyse préalable 21
4.1. Analyse de l'existant 21
4.2. Choix de la solution 22

4.3. Choix de la méthode de conception et de gestion de projet 23
4.4. Choix de l'outil de conception 25
4.5. Choix du langage de programmation 25
4.6. Choix du framework 26
4.7. Choix de l'environnement de développement 26
4.8. Choix du système de gestion de base de données 28
Chapitre 5 : Conception du projet 29
5.1. Présentation de la méthode de conception 29
5.2. Dictionnaire des données 29
5.3. Règles de gestion 30
5.4. Diagramme de cas d'utilisation 30
5.5. Description textuelle des cas d'utilisation 32
5.6. Diagramme de séquence système 32
5.7. Modèle de domaine 34
Chapitre 6 : Conception détaillée 36
6.1. Architecture de l'application 36
6.2. Diagrammes de séquence de conception 38
6.3. Diagramme de classes 45
6.4. Diagramme de paquetage 47
6.5. Diagramme de déploiement 47
PARTIE 3 : MISE EN ŒUVRE ET RÉSULTATS 49
Chapitre 7 : Mise en œuvre 50
7.1. Architecture du système 50
7.2. Configuration des outils de conception 51
7.3. Génération de la base de données 51
7.4. Configuration de l'environnement de développement 51
7.5. Extraits de code des fonctionnalités principales 51

7.6. Difficultés rencontrées 55
Chapitre 8 : Présentation de l'application 57
8.1. Espace public 57
8.2. Environnement matériel et déploiement logiciel 60
8.3. Authentification 61
8.4. Espace Présidence 62
8.5. Espace Établissement 63
8.6. Espace Décideur 64
8.7. Synthèse annuelle, notifications et exports 65
8.8. Responsive design, thème et accessibilité 66
Chapitre 9 : Évaluation et suggestions 67
9.1. Évaluation de l'application 67
9.2. Contributions académiques et professionnelles 69
9.3. Limitations de l'étude et pistes de recherche futures 69
CONCLUSION GÉNÉRALE 70
Bilan 70
Perspectives 70
BIBLIOGRAPHIE 71
WEBOGRAPHIE 72
ANNEXES 73
Annexe A : Synthèse des règles de gestion 73
Annexe B : Extraits de code complémentaires 74
Annexe C : Résultats d'exécution des tests 74
Table de matière 75